

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KỸ THUẬT PHẦN MỀM

XÂY DỰNG ỨNG DỤNG NHẬN DẠNG HOÁ ĐƠN
SỬ DỤNG MULTIMODAL LARGE LANGUAGE MODEL

GVHD : TS. Giang Thành Trung

Sinh viên : Trần Văn Lộc

Mã số sinh viên : 2021605411

Hà Nội – 2025

LỜI CẢM ƠN

Lời đầu tiên, em xin chân thành cảm ơn quý Thầy, Cô Trường Công nghệ thông tin và Truyền thông, Trường Đại học Công nghiệp đã hướng dẫn, chỉ dạy cho em những kiến thức chuyên ngành vô cùng quý giá. Những kiến thức đó đã hỗ trợ em rất nhiều trong quá trình tự học và nghiên cứu để hoàn thiện đồ án. Nhờ sự chỉ dạy tận tình đó mà em mới có thể áp dụng các kỹ năng và hiểu biết của bản thân để hoàn thành đồ án tốt nghiệp một cách trọn vẹn.

Trong đó, em xin trân trọng cảm ơn thầy Giang Thành Trung đã tận tình chỉ dạy, cầm tay chỉ việc và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập và hoàn thành đồ án tốt nghiệp.

Trong quá trình hoàn thành đồ án tốt nghiệp không thể tránh khỏi những thiếu sót. Kính mong các quý thầy cô chỉ bảo và đóng góp ý kiến để đề tài đồ án của em được hoàn thiện hơn. Lời cuối cùng em xin chúc các quý thầy, cô và các bạn luôn dồi dào sức khỏe và thành công trong cuộc sống. Xin chúc những điều tốt đẹp nhất sẽ luôn đồng hành cùng mọi người!

Em xin chân thành cảm ơn.

Sinh viên thực hiện

Trần Văn Lộc

MỤC LỤC

LỜI CẢM ƠN	i
MỤC LỤC	ii
DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT ...	iv
DANH MỤC HÌNH ẢNH.....	v
MỞ ĐẦU	1
CHƯƠNG 1 CƠ SỞ LÝ THUYẾT	3
1.1. Mô hình tiến trình phần mềm	3
1.2. Kiến trúc phần mềm, mô hình lập trình	4
1.3. Mô hình ngôn ngữ lớn (LLM)	7
1.3.1. Tổng quan.....	7
1.3.2. Các thành phần cơ bản	8
1.3.3. Cách hoạt động	8
1.3.4. Ứng dụng.....	9
1.4. Mô hình Ngôn ngữ Đa phương thức (MLLM).....	10
1.5. Tổng quan về công nghệ sử dụng	11
1.5.1. Vintern-1B-v2.....	11
1.5.2. Google colab	16
1.5.3. Ngrok.....	17
CHƯƠNG 2 PHÂN TÍCH THIẾT KẾ HỆ THỐNG.....	18
2.1. Phân tích hệ thống.....	18
2.1.1. Giới thiệu chung	18
2.1.2. Yêu cầu chức năng.....	18
2.1.3. Yêu cầu phi chức năng.....	19
2.2. Đặc tả hệ thống	19
2.2.1. Biểu đồ Usecase.....	19
2.2.2. Use case Gửi yêu cầu	20
2.2.3. Use case Chọn hình ảnh.....	21
2.2.4. Use case Xuất file Excel	22

2.2.5. Use case Sửa dữ liệu	23
2.3. Thiết kế giao diện.....	24
CHƯƠNG 3 CÀI ĐẶT VÀ KIỂM THỬ	27
3.1. Cài đặt và chạy chương trình.....	27
3.1.1. Triển khai backend LLM Vision trên Google Colab	27
3.1.2. Triển khai trang trình phía client trên VS Code.....	30
3.1.3. Kết quả.....	34
3.2. Kiểm thử	37
3.2.1. Chuẩn bị dữ liệu kiểm thử	37
3.2.2. Mục tiêu kiểm thử	37
3.2.3. Phạm vi kiểm thử	37
3.2.4. Phương pháp kiểm thử	38
3.2.5. Kết quả kiểm thử.....	38
3.2.6. Kết luận	41
KẾT LUẬN	42
4.1. Kiến thức đạt được	42
4.2. Kỹ năng đạt được.....	42
4.3. Bài học kinh nghiệm	42
TÀI LIỆU THAM KHẢO	43
PHỤ LỤC.....	44

DANH MỤC CÁC THUẬT NGỮ, KÝ HIỆU VÀ CÁC CHỮ VIẾT TẮT

Chữ cái/thuật ngữ viết tắt	Từ đầy đủ
LLM	Large Language Model
MLLM	Multimodal Large Language Model
OCR	Optical character recognition

DANH MỤC HÌNH ẢNH

Hình 1.1. Mô hình thác nước.....	3
Hình 1.2. Kiến trúc phần mềm.....	5
Hình 1.3. Kiến trúc tổng thể của Vintern-1B-v2	13
Hình 1.4. Google Colab	16
Hình 1.5. Ngrok.....	17
Hình 2.1. Biểu đồ Usecase.....	19
Hình 2.2. Biểu đồ trình tự Usecase Gửi yêu cầu	20
Hình 2.3. Biểu đồ trình tự Usecase Chọn hình ảnh	21
Hình 2.5. Biểu đồ trình tự Usecase Xuất file Excel.....	22
Hình 2.4. Biểu đồ trình tự Usecase Sửa dữ liệu	23
Hình 2.6. Thiết kế giao diện màn hình chính	24
Hình 2.7. Thiết kế giao diện thông báo Nhập đường dẫn ảnh	24
Hình 2.8. Thiết kế giao diện thông báo Chọn ảnh từ thiết bị	25
Hình 2.9. Thiết kế giao diện thông báo Xuất file thành công	25
Hình 2.10. Thiết kế giao diện thông báo Xuất file không thành công	26
Hình 3.1. Nguyên lý hoạt động của chương trình	27
Hình 3.1. Giao diện chính.....	34
Hình 3.2. Giao diện hiển thị kết quả	35
Hình 3.3. Cửa sổ chọn hình ảnh từ thiết bị.....	35
Hình 3.4. Cửa sổ thông báo khi để đường dẫn ảnh trống	36
Hình 3.5. Cửa sổ thông báo lỗi kết nối server	36
Hình 3.6. Cửa sổ thông báo nếu dữ liệu trong bảng trống.....	36
Hình 3.7. Cửa sổ thông báo xuất file thành công	37

MỞ ĐẦU

1. Lý do chọn đề tài

Vấn đề hiện tại:

- Quy trình xử lý hóa đơn thủ công tốn thời gian, công sức và dễ xảy ra sai sót.
- Nhu cầu tự động hóa quy trình này trong các doanh nghiệp ngày càng tăng để nâng cao hiệu quả và giảm chi phí.
- Các phương pháp nhận dạng hóa đơn truyền thống có thể gặp khó khăn với sự đa dạng về định dạng và chất lượng hình ảnh hóa đơn.

Tiềm năng của Multimodal Large Language Model (MLLM):

- Khả năng xử lý đồng thời thông tin từ nhiều modal (văn bản, hình ảnh) giúp MLLM hiểu hóa đơn một cách toàn diện hơn.
- Sức mạnh của các mô hình ngôn ngữ lớn trong việc hiểu ngữ cảnh và trích xuất thông tin phức tạp.
- Tiềm năng vượt trội so với các phương pháp truyền thống trong việc xử lý hóa đơn không theo cấu trúc hoặc có chất lượng kém.

Tính mới và đóng góp của đề tài:

- Ứng dụng MLLM trong bài toán nhận dạng hóa đơn có thể là một hướng nghiên cứu mới hoặc chưa được khai thác sâu.
- Đề tài có thể đóng góp vào việc phát triển các giải pháp tự động hóa quy trình kế toán và quản lý tài chính hiệu quả hơn cho doanh nghiệp.

2. Mục tiêu nghiên cứu:

- Xây dựng một ứng dụng có khả năng nhận dạng và trích xuất thông tin chính xác từ nhiều định dạng hóa đơn khác nhau.
- Đánh giá hiệu suất của ứng dụng đã xây dựng dựa trên các bộ dữ liệu hóa đơn thực tế.

- So sánh hiệu suất của ứng dụng sử dụng MLLM với các phương pháp nhận dạng hóa đơn truyền thống.

- Nghiên cứu khả năng tùy chỉnh và mở rộng của ứng dụng cho các yêu cầu cụ thể của doanh nghiệp.

3. Đối tượng và phạm vi nghiên cứu:

- Mô hình Multimodal Large Language Model: Vintern-1B-v2
- Các phương pháp và kỹ thuật tiền xử lý hình ảnh và văn bản hóa đơn.
- Các công cụ và thư viện hỗ trợ phát triển ứng dụng: Google Colab, Ngrok, Visual Studio Code, Request, Tkinter, OS, Openpyxl.
- Ứng dụng được vận dụng trong thực tế tại các doanh nghiệp.

4. Kết quả mong muốn

- Một mô hình MLLM đã được huấn luyện hoặc tinh chỉnh cho bài toán nhận dạng hóa đơn.
- Một ứng dụng demo có khả năng tải lên hình ảnh hóa đơn và trả về thông tin đã được nhận dạng (ví dụ: mã số thuế, tên người bán, tên hàng hóa, số lượng, đơn giá, thành tiền, tổng tiền...).
- Báo cáo đánh giá chi tiết về hiệu suất của mô hình và ứng dụng (độ chính xác, thời gian xử lý...).
- Mã nguồn của ứng dụng và mô hình

5. Cấu trúc của báo cáo:

Ngoài các phần Mở đầu, Kết luận và Tài liệu tham khảo, báo cáo đồ án được bố cục thành 3 chương chính như sau:

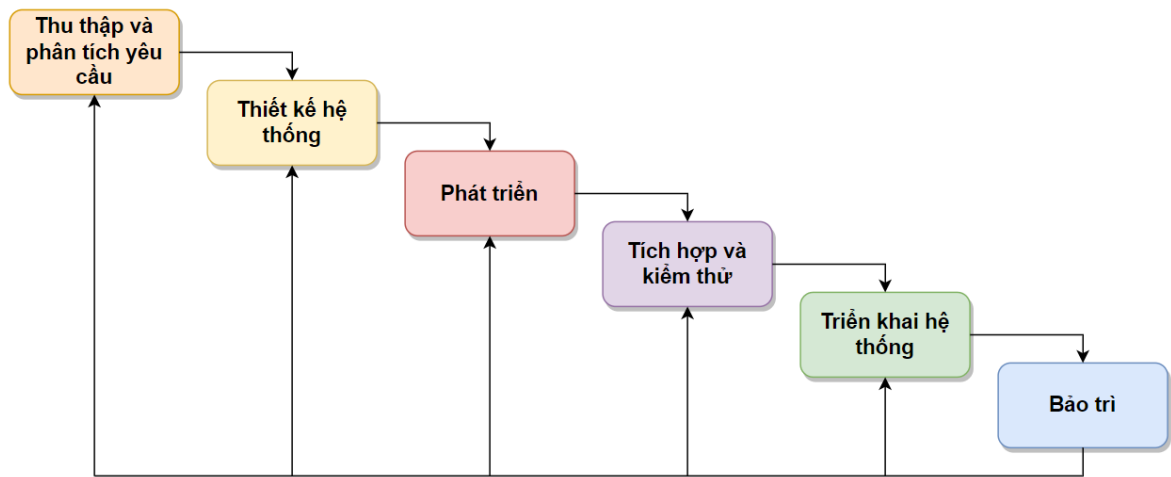
- Chương 1: Trình bày tổng quan về cơ sở lý thuyết
- Chương 2: Trình bày tổng quan phân tích, thiết kế hệ thống
- Chương 3: Trình bày về quá trình cài đặt và kiểm thử

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT

1.1. Mô hình tiến trình phần mềm

Mô hình quy trình phát triển dự án này sử dụng là mô hình thác nước. *Mô hình thác nước* (Waterfall model) là mô hình quy trình phát triển phần mềm đầu tiên được giới thiệu. Nó cũng được gọi là mô hình vòng đời tuần tự tuyến tính. Trong mô hình thác nước, mỗi giai đoạn phải được hoàn thành trước khi giai đoạn tiếp theo có thể bắt đầu và không có sự chồng chéo trong các giai đoạn.



Hình 1.1. Mô hình thác nước

Các giai đoạn trong quy trình thác nước gồm:

- Thu thập và phân tích yêu cầu: Các yêu cầu được ghi lại trong giai đoạn này.
- Thiết kế hệ thống: Xác định các yêu cầu phần cứng và kiến trúc tổng thể.
- Phát triển: Mỗi đơn vị được phát triển và kiểm tra chức năng của nó.
- Tích hợp và Kiểm thử: Tất cả các đơn vị được tích hợp vào một hệ thống và toàn hệ thống được kiểm tra lỗi.
- Triển khai hệ thống: Sản phẩm được triển khai trong môi trường khách hàng/thị trường.

- Bảo trì: Bảo trì được thực hiện để mang lại những thay đổi tích cực.

Ưu điểm và nhược điểm của mô hình quy trình thác nước:

- *Ưu điểm :*

- Đơn giản, dễ hiểu và dễ sử dụng
- Mỗi giai đoạn có các phân phối cụ thể và một quy trình xem xét
- Các giai đoạn được xử lý và hoàn thành cùng một lúc
- Các giai đoạn được xác định rõ ràng
- Các mốc quan trọng được hiểu rõ
- Dễ dàng sắp xếp các công việc
- Quá trình và kết quả được ghi chép đầy đủ

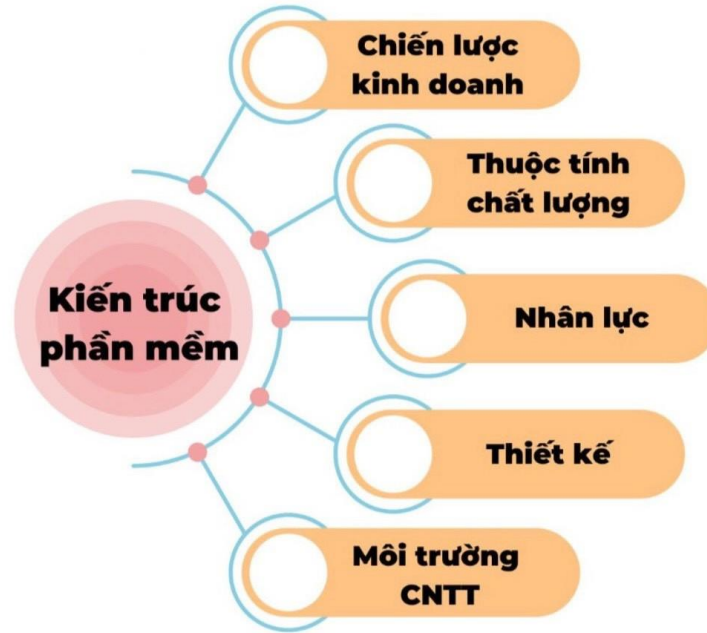
- *Nhược điểm:*

- Lượng rủi ro cao và không chắc chắn
- Rất khó để đo lường sự tiến bộ trong các giai đoạn
- Không thể đáp ứng các yêu cầu thay đổi
- Không phải là mô hình tốt cho các dự án phức tạp và hướng đối tượng.

1.2. Kiến trúc phần mềm, mô hình lập trình

Kiến trúc phần mềm là tập hợp các cấu trúc cần thiết để lập trình nên hệ thống phần mềm, bao gồm các phần tử, mối quan hệ giữa chúng và các thuộc tính của chúng.

Kiến trúc phần mềm cung cấp một nền tảng vững chắc để Software Engineer xây dựng nên phần mềm. Mỗi dự án phần mềm sẽ có một bản tóm tắt, và kiến trúc phần mềm là bước đầu tiên để thực hiện bản tóm tắt đó. Kiến trúc và thiết kế phần mềm bao gồm một số yếu tố như: Chiến lược kinh doanh, thuộc tính chất lượng, nhân lực, thiết kế và môi trường CNTT.



Hình 1.2. Kiến trúc phần mềm

Mục tiêu chính của kiến trúc là xác định các yêu cầu ảnh hưởng đến cấu trúc của ứng dụng. Một kiến trúc được bố trí hợp lý giúp giảm rủi ro kinh doanh liên quan đến việc xây dựng giải pháp kỹ thuật và xây dựng cầu nối giữa yêu cầu kinh doanh và kỹ thuật.

Một số mục tiêu khác của kiến trúc phần mềm có thể kể đến như sau:

- Phơi bày cấu trúc của hệ thống, nhưng ẩn các chi tiết triển khai của nó.
- Nhận ra tất cả các trường hợp sử dụng và tình huống.
- Cố gắng giải quyết các yêu cầu của các bên liên quan khác nhau.
- Xử lý cả các yêu cầu về chức năng và chất lượng.
- Giảm mục tiêu sở hữu và nâng cao vị thế thị trường của tổ chức.
- Cải thiện chất lượng và chức năng do hệ thống cung cấp.

Mối quan hệ giữa kiến trúc phần mềm và thiết kế phần mềm: Kiến trúc phần mềm thể hiện cấu trúc của một hệ thống trong khi các chi tiết triển khai bị ẩn đi. Kiến trúc cũng tập trung vào cách các yếu tố và thành phần trong hệ thống tương tác với nhau. Thiết kế phần mềm đi sâu hơn vào các chi tiết triển

khai của hệ thống. Mỗi quan tâm về thiết kế bao gồm việc lựa chọn cấu trúc dữ liệu và thuật toán hoặc chi tiết triển khai của các thành phần riêng lẻ.

Vai trò của kiến trúc phần mềm: Kiến trúc đóng vai trò như bản thiết kế cho một hệ thống. Nó cung cấp một cái nhìn tổng thể để quản lý độ phức tạp của hệ thống, thiết lập một cơ chế giao tiếp và phối hợp giữa các thành phần.

Một số vai trò chính của kiến trúc phần mềm bao gồm:

- Giúp các bên liên quan hình dung, hiểu và phân tích hệ thống.
- Kiến trúc phần mềm giúp phác họa cấu trúc của phần mềm.
- Nó có thể được sử dụng để dự trù kinh phí trong một dự án trước khi chúng được thực hiện.
- Nó được sử dụng làm cơ sở để đưa ra các quyết định cốt lõi nhằm đảm bảo chất lượng công việc.

Các mẫu kiến trúc phần mềm phổ biến:

- Mô hình phân lớp: Các thành phần trong mô hình kiến trúc phần mềm được tách thành các lớp nhiệm vụ con và chúng được sắp xếp chồng lên nhau. Mỗi lớp có các nhiệm vụ duy nhất để thực hiện và tất cả các lớp đều độc lập với nhau. Vì mỗi lớp đều độc lập nên người ta có thể sửa đổi mã bên trong một lớp mà không ảnh hưởng đến các lớp khác.

- Mô hình Client – Server: Mô hình này có hai thực thể chính: một máy chủ và nhiều máy khách. Ở đây máy chủ có tài nguyên (dữ liệu, tệp hoặc dịch vụ) và máy khách yêu cầu máy chủ cung cấp một tài nguyên cụ thể. Sau đó, máy chủ xử lý yêu cầu và phản hồi lại tương ứng.

- Mô hình Event – Driven: Khi người dùng thực hiện hành động trong ứng dụng được xây dựng bằng cách tiếp cận EDA (Exploratory Data Analysis – Phân tích Khám phá Dữ liệu), sự thay đổi trạng thái sẽ xảy ra và phản ứng được tạo ra được gọi là sự kiện.

- Mô hình Microkernel: có hai thành phần chính: một hệ thống cốt lõi và các mô-đun (Module) plug-in. Hệ thống cốt lõi xử lý các hoạt động cơ bản của

ứng dụng. Các mô-đun plug-in xử lý các chức năng mở rộng (như các tính năng bổ sung) và xử lý tùy chỉnh.

- Mô hình Microservices: là tập hợp các dịch vụ nhỏ được kết hợp để tạo thành ứng dụng thực tế. Thay vì xây dựng một ứng dụng lớn hơn, các chương trình nhỏ được xây dựng cho mọi dịch vụ (chức năng) của ứng dụng một cách độc lập. Và những chương trình nhỏ đó được gộp lại với nhau để trở thành một ứng dụng chính thức. Các mô-đun trong ứng dụng của các mẫu Microservices được ghép nối một cách lỏng lẻo. Vì vậy, bạn có thể dễ dàng sửa đổi và mở rộng.

1.3. Mô hình ngôn ngữ lớn (LLM)

1.3.1. Tổng quan

Mô hình Ngôn ngữ lớn (Large Language Model - LLM) là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản khổng lồ, có khả năng hiểu, tạo và xử lý ngôn ngữ tự nhiên. Sự phát triển của LLM đã có những bước tiến vượt bậc trong những năm gần đây nhờ vào sự gia tăng của sức mạnh tính toán và lượng dữ liệu sẵn có.

Từ đó cho phép mô hình nắm bắt cú pháp, ngữ nghĩa, cấu trúc ngôn từ. Thậm chí một số khía cạnh của kiến thức chung để xử lý và tạo ra văn bản. Mô hình không chỉ có khả năng sinh ra văn bản tự nhiên. Mà còn có thể được ứng dụng trong nhiều lĩnh vực. Như xử lý ngôn ngữ, trả lời câu hỏi, dịch thuật, tóm tắt văn bản. Thậm chí là tạo nội dung tự động.

Tuy nhiên, ta cũng cần lưu ý rằng Large Language Models không phải là hoàn hảo và nó có thể phản ánh các đặc điểm cũng như giới hạn của dữ liệu được huấn luyện. Điều này đặt ra những thách thức về đạo đức và quản lý chất lượng thông tin khi sử dụng AI LLM trong các ứng dụng thực tế.

1.3.2. Các thành phần cơ bản

Large Language Models là một hệ thống phức tạp kết hợp nhiều layer neural network (mạng nơron) riêng biệt. Các thành phần hoạt động phối hợp với nhau để có thể xử lý văn bản đầu vào và tạo ra nội dung như mong muốn:

- **Embedding layer:** Là lớp đầu tiên của LLM. Có chức năng chính là biểu diễn từng từ vựng trong văn bản đầu vào thành các vector số học biểu diễn nhiều chiều (high-dimensional). Mang thông tin về ngữ nghĩa và cú pháp của từ hoặc token đó trong câu.

- **Feedforward layer:** Viết tắt là FFN, layer này gồm nhiều lớp được kết nối với nhau. Áp dụng các phép biến đổi phi tuyến tính trên đầu ra của các lớp trước đó để tạo ra các biểu diễn từ hoặc đoạn văn có chiều sâu và giàu thông tin hơn.

- **Recurrent layer:** Hoạt động theo cách xử lý thông tin tuần tự và tạo ra các biểu diễn từ có tính tuần tự và phụ thuộc vào ngữ cảnh. Nó giúp mô hình hiểu và nắm bắt mối quan hệ phức tạp giữa các từ trong câu để tạo ra chuỗi văn bản có ý nghĩa.

- **Attention mechanism:** Cơ chế này giúp mô hình ngôn ngữ lớn tập trung vào các phần quan trọng của đầu vào trong khi tạo đầu ra. Nó cho phép AI LLM chú ý đến các phần khác nhau của ngữ cảnh và ưu tiên xử lý các thông tin liên quan hơn trước.

1.3.3. Cách hoạt động

Mô hình ngôn ngữ lớn hoạt động bằng cách sử dụng mạng nơron sâu, thường là dựa trên kiến trúc transformer. Tuân theo quy trình bao gồm mã hóa đầu vào, giải mã và dự đoán đầu ra. Nó nhúng từ, biểu diễn mỗi từ dưới dạng vector số. Và sử dụng lớp transformer để hiểu mối quan hệ giữa từ.

Mô hình ngôn ngữ lớn thực hiện các phép toán tuyến tính và phi tuyến tính. Thông qua các lớp feedforward. Và sử dụng cơ chế attention để tập trung vào các phần quan trọng. Thông qua quá trình huấn luyện và fine-tuning. Mô

hình học cách hiểu và tạo ra ngôn ngữ tự nhiên. Từ đó dự đoán từ tiếp theo trong chuỗi văn bản. Có thể thực hiện nhiều nhiệm vụ như tạo văn bản mới, trả lời câu hỏi và dịch ngôn ngữ.

1.3.4. Ứng dụng

Large Language Models có nhiều ứng dụng quan trọng trong các lĩnh vực thực tế khác nhau. LLM có thể ứng dụng trong dịch thuật, hoàn thiện câu, phân tích tâm lý, trả lời câu hỏi,... Một số ứng dụng phổ biến như:

- **Tạo văn bản tự động:** Tạo văn bản sáng tạo, bài luận hoặc nội dung Marketing, phát triển nội dung, tóm tắt tin tức cho trang web, blog hoặc các ứng dụng khác.

- **Dịch ngôn ngữ:** Hỗ trợ dịch văn bản từ ngôn ngữ này sang ngôn ngữ khác. Mô hình giúp giao tiếp đa ngôn ngữ trong ứng dụng và trang web.

- **Chatbot và Trợ lý ảo:** Trả lời câu hỏi trên các diễn đàn, trang web Q&A. Hỗ trợ người dùng tìm kiếm thông tin trên internet và tương tác người-máy.

- **Học máy và NLP (Natural Language Processing):** Tích hợp vào ứng dụng và dự án học máy Machine Learning để hiểu và xử lý văn bản tự nhiên. Cũng như phân loại tin tức, phân đoạn ý kiến hay nhận diện thư rác. Phát triển các ứng dụng NLP như chatbot hoặc giao diện người dùng thông minh.

- **Giáo dục và hỗ trợ học tập:** Giúp tạo tài liệu giáo trình và bài giảng. Hỗ trợ học sinh, sinh viên trong việc nắm bắt kiến thức và trả lời câu hỏi.

- **Y tế và y học:** Hỗ trợ việc phân tích và tổ chức thông tin y tế từ văn bản và tài liệu y học. Góp phần phát triển ứng dụng hỗ trợ chẩn đoán và tư vấn y tế.

- **Phát triển ứng dụng và trò chơi:** LLM được tích hợp và ứng dụng di động và trò chơi để cải thiện trải nghiệm của người dùng.

- **Quản lý dữ liệu và thông tin:** Hỗ trợ tổ chức và tìm kiếm thông tin trong doanh nghiệp cũng như quản lý dữ liệu dự án.

1.4. Mô hình Ngôn ngữ Đa phương thức (MLLM)

Mô hình Ngôn ngữ Đa phương thức (Multimodal Large Language Model - MLLM) là sự kết hợp giữa khả năng xử lý ngôn ngữ của LLM và khả năng hiểu các modalities khác như hình ảnh, âm thanh, video, v.v. MLLM cho phép máy tính không chỉ "đọc" và "viết" mà còn có thể "nhìn", "nghe", tạo ra sự hiểu biết toàn diện hơn về thế giới.

Trong thế giới thực, thông tin thường tồn tại dưới nhiều dạng khác nhau. Để AI có thể tương tác và hiểu thế giới một cách tự nhiên, khả năng xử lý đa phương thức là rất quan trọng. MLLM giúp thu hẹp khoảng cách giữa các modalities khác nhau.

Các phương pháp kết hợp thông tin đa phương thức:

- **Early Fusion:** Kết hợp dữ liệu từ các modalities khác nhau ở giai đoạn đầu của mô hình.
- **Late Fusion:** Xử lý từng modality riêng biệt bằng các mô hình chuyên biệt, sau đó kết hợp kết quả ở giai đoạn cuối.
- **Cross-attention:** Sử dụng cơ chế attention để cho phép mô hình từ một modality "chú ý" đến các phần quan trọng trong modality khác.
- **Projectors:** Các lớp tuyến tính hoặc mạng nơ-ron nhỏ được sử dụng để ánh xạ biểu diễn từ một modality (ví dụ: đặc trưng hình ảnh từ Vision Encoder) sang không gian biểu diễn của modality khác (ví dụ: không gian embedding của Language Model).

Các kiến trúc MLLM phổ biến:

- **Frozen Vision Encoder + Fine-tuned LLM:** Sử dụng một mô hình thị giác đã được huấn luyện sẵn (frozen), trích xuất đặc trưng hình ảnh và đưa vào LLM để tinh chỉnh.
- **Frozen LLM + Fine-tuned Vision Encoder:** Ngược lại, giữ LLM cố định và tinh chỉnh mô hình thị giác hoặc bộ chiếu (projector) để kết nối với LLM.

- **Full Fine-tuning:** Tinh chỉnh toàn bộ cả mô hình thị giác và mô hình ngôn ngữ cùng lúc.

- **Adapter-based Methods:** Sử dụng các lớp Adapter nhỏ được thêm vào giữa các lớp của mô hình thị giác và ngôn ngữ để kết nối chúng mà không cần tinh chỉnh toàn bộ mô hình gốc.

1.5. Tổng quan về công nghệ sử dụng

1.5.1. Vintern-1B-v2

1.5.1.1. Giới thiệu

Vintern-1B-v2 là một mô hình ngôn ngữ lớn đa phương thức (Multimodal Large Language Model - MLLM) được phát triển bởi nhóm nghiên cứu Fifth Civil Defender (5CD-AI) dựa trên nền tảng của các mô hình InternVL từ OpenGVLab, tối ưu hóa cho ngôn ngữ tiếng Việt. Mô hình này nổi bật với khả năng xử lý kết hợp cả thông tin dạng văn bản và hình ảnh, mang lại nhiều ứng dụng tiềm năng, đặc biệt là trong các tác vụ đòi hỏi hiểu biết về nội dung trực quan.

Vintern-1B-v2 là phiên bản cải tiến, kế thừa và phát huy những điểm mạnh từ phiên bản trước đó. Mô hình có kích thước 1 tỷ tham số, được thiết kế để hoạt động hiệu quả ngay cả trên các thiết bị có tài nguyên hạn chế, mở rộng khả năng triển khai trong nhiều môi trường khác nhau.

1.5.1.2. Quá trình phát triển

- **Mô hình cơ sở:** Bắt đầu từ InternVL2.5-1B, một mô hình đa phương thức từ OpenGVLab, nổi tiếng với hiệu suất mạnh mẽ trong các tác vụ kết hợp thị giác và ngôn ngữ.

- **Tinh chỉnh trên dữ liệu tiếng Việt:** Mô hình được tinh chỉnh thêm trên tập dữ liệu Viet-ShareGPT-4o-Text-VQA, bao gồm các cặp câu hỏi-đáp dựa trên hình ảnh, được thiết kế đặc biệt cho tiếng Việt.

Các phiên bản trước:

- **ColVintern-1B-v1**: phiên bản đầu tiên dòng 1B, tập trung vào truy xuất thông tin từ cả văn bản và hình ảnh bằng cách tích hợp pipeline Colpali. Colpali sử dụng mô hình PaliGemma để tạo embedding đa vector trực tiếp từ hình ảnh tài liệu, giúp loại bỏ OCR phức tạp và cải thiện độ chính xác truy xuất. Mô hình này hỗ trợ RAG, ưu tiên tiếng Việt, và đạt hiệu suất cao với chỉ 1 tỷ tham số, hướng tới sự nhẹ và hiệu quả hơn so với các phiên bản trước.

- **Vintern-1B-v2**: là phiên bản nâng cấp của ColVintern-1B-v1, cải thiện đáng kể hiệu suất trên các benchmark, nâng cao khả năng xử lý đa phương thức bằng tiếng Việt. Mô hình này kết hợp Qwen2-0.5B-Instruct và InternViT-300M-448px, với 1 tỷ tham số và độ dài ngữ cảnh (context length) 4096, được tinh chỉnh trên hơn 3 triệu cặp câu hỏi-trả lời cho OCR, VQA và trích xuất tài liệu. Những cải tiến này giúp Vintern-1B-v2 vượt xa nhiệm vụ truy xuất tài liệu, mở rộng khả năng hiểu thông tin thị giác và ngôn ngữ.

1.5.1.3. Cấu trúc mô hình

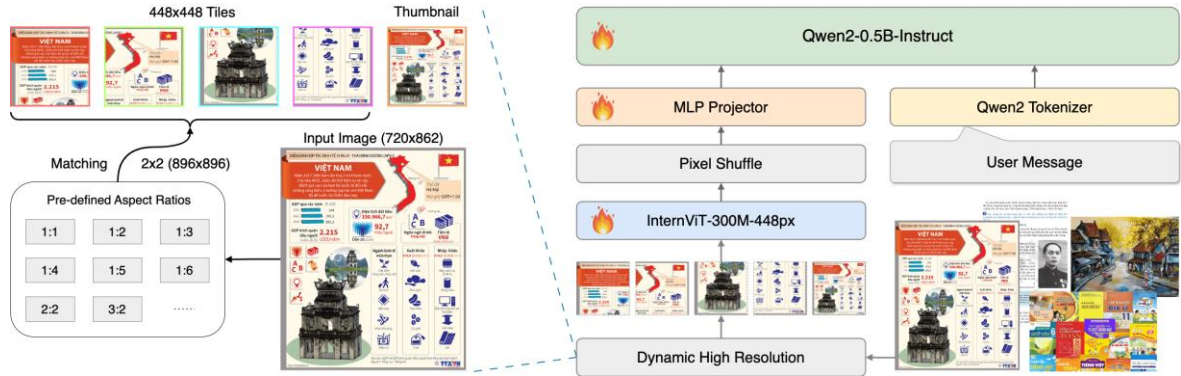
Vintern-1B-v2 sử dụng kiến trúc "ViT-MLP-LLM", một thiết kế phổ biến và hiệu quả cho các MLLM. Kiến trúc này tương tự như nhiều MLLM nổi tiếng khác. ViT-MLP-LLM của Vintern-1B-v2 sẽ gồm 3 phần như sau:

- Mô hình thị giác InternViT-300M-448px đóng vai trò là thành phần xử lý đầu vào hình ảnh. Mô hình này được huấn luyện chất lọc kiến thức từ mô hình InternViT-6B-448px-V1-5 mạnh mẽ và có khả năng xử lý hình ảnh với độ phân giải 448x448 pixel.

- Mô hình ngôn ngữ Qwen2-0.5B-Instruct chịu trách nhiệm xử lý và tạo ra đầu ra văn bản. Các mô hình Qwen2 được biết đến với khả năng đa ngôn ngữ mạnh mẽ, bao gồm cả tiếng Việt. Phiên bản 0.5 tỷ tham số này góp phần tăng hiệu suất của Vintern 1B-V2.

- Một thành phần quan trọng khác là MLP (Multi-Layer Perceptron) projector, có vai trò kết nối mô hình thị giác và mô hình ngôn ngữ. MLP projector hoạt động như một cơ chế để điều chỉnh các biểu diễn đặc trưng từ

InternViT và Qwen2, cho phép chúng tương tác hiệu quả với nhau. Thành phần này rất quan trọng để kích hoạt chức năng đa phương thức, cho phép mô hình hiểu mối quan hệ giữa thông tin thị giác và văn bản.



Hình 1.3. Kiến trúc tổng thể của Vintern-1B-v2

Hình ảnh đầu vào được xử lý bởi mô-đun Độ phân giải Cao Động (Dynamic High Resolution), trong đó hình ảnh được chia thành các phần nhỏ có kích thước 448x448 pixel cùng với một ảnh thu nhỏ (thumbnail). Các phần hình ảnh này sau đó được đưa vào mô hình InternViT-300M-448px để trích xuất đặc trưng thị giác. Một bước xáo trộn điểm ảnh (Pixel Shuffle) cũng được áp dụng trước khi truyền dữ liệu vào bộ chiếu MLP (MLP projector) nhằm căn chỉnh với các embedding của mô hình ngôn ngữ lớn Qwen2-0.5B-Instruct. Mô hình này tiếp nhận các token thị giác đã được căn chỉnh cùng với câu hỏi liên quan và tạo ra câu trả lời tương ứng.

Với những khả năng của mình, Vintern-1B-v2 có thể được ứng dụng trong nhiều lĩnh vực như:

- Trích xuất thông tin từ hóa đơn, chứng từ, văn bản hành chính.
- Xử lý và hiểu nội dung sách giáo khoa, tài liệu kỹ thuật có hình ảnh minh họa.
- Phát triển các ứng dụng hỏi đáp về nội dung hình ảnh bằng tiếng Việt.
- Hỗ trợ người dùng tương tác với tài liệu quét hoặc hình ảnh chứa văn bản.

1.5.1.4. Lý do lựa chọn

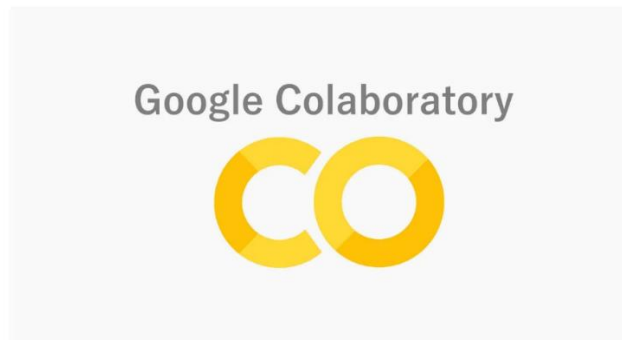
	Vintern-1B	Phương pháp truyền thống
Khả năng tích hợp đa phương thức	Được thiết kế để xử lý đồng thời và tích hợp thông tin từ nhiều modalities (hình ảnh và văn bản). Mô hình có thể "nhìn" hình ảnh và "đọc" văn bản, sau đó suy luận dựa trên cả hai nguồn thông tin. Điều này cho phép nó giải quyết các tác vụ phức tạp như trả lời câu hỏi về hình ảnh (VQA) hoặc hiểu tài liệu có cấu trúc phức tạp (ví dụ: hóa đơn có cả chữ và bố cục).	Thường xử lý từng modality một cách riêng biệt. Ví dụ, để xử lý một hóa đơn, sẽ cần một hệ thống OCR để trích xuất văn bản, sau đó một hệ thống xử lý ngôn ngữ tự nhiên (NLP) riêng biệt để hiểu ý nghĩa của văn bản đó. Việc kết hợp thông tin giữa các hệ thống riêng biệt này thường phức tạp, dễ xảy ra lỗi và khó duy trì ngữ cảnh giữa hình ảnh và văn bản.
Hiểu ngữ cảnh và suy luận	Nhờ kiến trúc Transformer và quá trình huấn luyện trên lượng dữ liệu lớn, Vintern-1B có khả năng hiểu ngữ cảnh sâu sắc hơn và thực hiện các suy luận phức tạp hơn. Ví dụ, khi hỏi "Giá tiền của sản phẩm X là bao nhiêu?" trên một hóa đơn, mô hình có thể không chỉ đọc được số tiền mà còn hiểu được "sản phẩm X" là gì trong hình ảnh và liên kết đúng với giá trị tương ứng.	Các hệ thống truyền thống thường dựa trên các quy tắc (rule-based) hoặc các mô hình học máy nông (shallow machine learning) được huấn luyện trên dữ liệu hạn chế. Khả năng suy luận và hiểu ngữ cảnh của chúng thường bị giới hạn, khó mở rộng và dễ bị phá vỡ khi gặp các trường hợp ngoại lệ.

Khả năng tổng quát hóa	Có khả năng tổng quát hóa tốt hơn nhiều. Sau khi được huấn luyện trên một lượng lớn dữ liệu đa dạng, mô hình có thể thích nghi với các biến thể mới của dữ liệu hoặc các tác vụ tương tự mà không cần tinh chỉnh lại từ đầu.	Thường kém linh hoạt hơn. Một hệ thống OCR truyền thống có thể hoạt động tốt trên một loại phong chữ hoặc bố cục tài liệu cụ thể, nhưng sẽ gặp khó khăn khi gặp các loại mới. Mỗi tác vụ hoặc loại tài liệu mới thường đòi hỏi việc xây dựng và huấn luyện lại một mô hình riêng biệt.
Yêu cầu về dữ liệu huấn luyện	Mặc dù yêu cầu lượng dữ liệu khổng lồ cho giai đoạn tiền huấn luyện, nhưng sau đó, việc tinh chỉnh cho các tác vụ cụ thể có thể chỉ cần một lượng dữ liệu nhỏ hơn (ví dụ: vài trăm đến vài nghìn mẫu) để đạt được hiệu suất cao.	Các mô hình học máy truyền thống hoặc hệ thống dựa trên quy tắc thường yêu cầu việc gán nhãn dữ liệu rất chi tiết và tốn công sức cho từng tác vụ cụ thể.
Độ phức tạp trong phát triển và triển khai	Việc phát triển một MLLM từ đầu là cực kỳ phức tạp và tốn kém tài nguyên. Tuy nhiên, việc sử dụng các mô hình đã được tiền huấn luyện sẵn như Vintern-1B (thông qua fine-tuning hoặc prompting) giúp giảm đáng kể độ phức tạp và thời gian phát triển ứng dụng.	Có thể đơn giản hơn để phát triển cho một tác vụ rất cụ thể và đơn giản. Tuy nhiên, khi các yêu cầu trở nên phức tạp hơn hoặc cần tích hợp nhiều chức năng, độ phức tạp của việc kết hợp các hệ thống riêng biệt sẽ tăng lên nhanh chóng.

Bảng 1. So sánh Vintern-1B-V2 với mô hình truyền thống

1.5.2. Google colab

Google Colab, hay còn gọi là Colaboratory, là một dịch vụ miễn phí, dựa trên nền tảng đám mây của Google Research cho phép mọi người viết và thực thi mã Python thông qua trình duyệt web. Được xây dựng dựa trên Project Jupyter, Colab đặc biệt phù hợp cho các công việc liên quan đến học máy, phân tích dữ liệu và mục đích giáo dục.



Hình 1.4. Google Colab

Một lợi thế chính của Colab là môi trường không cần cài đặt, cho phép người dùng bắt đầu viết mã ngay lập tức mà không cần cài đặt hoặc cấu hình bất cứ điều gì trên máy tính cá nhân của họ. Nó cung cấp quyền truy cập miễn phí vào các tài nguyên tính toán, bao gồm GPU và TPU, rất cần thiết để tăng tốc các tác vụ đòi hỏi nhiều tính toán như huấn luyện các mô hình học máy.

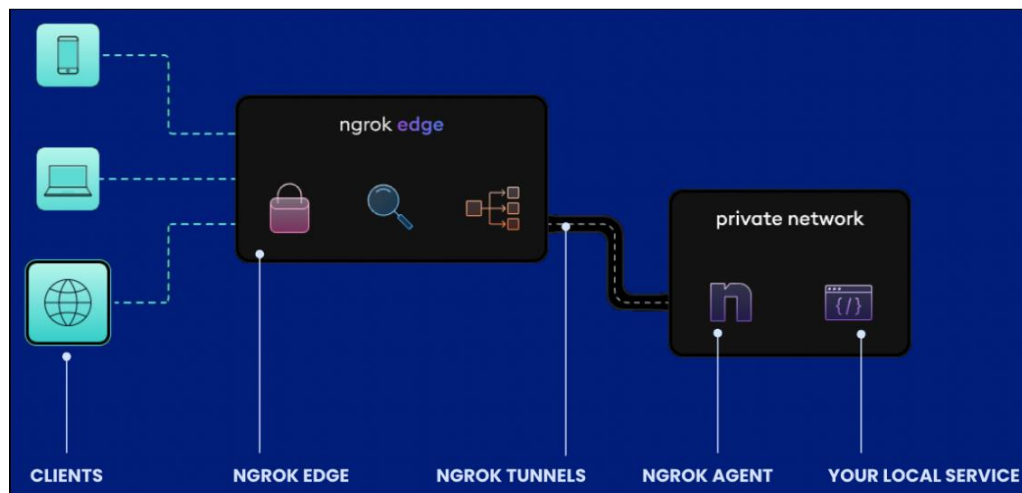
Colab tạo điều kiện thuận lợi cho việc cộng tác, cho phép nhiều người dùng cùng làm việc trên cùng một sổ ghi chép (notebook) cùng lúc, tương tự như cách người dùng cộng tác trên Google Docs. Nó cũng tích hợp liền mạch với Google Drive và GitHub, đơn giản hóa quá trình truy cập và lưu tệp cũng như quản lý kho mã nguồn.

Nền tảng này được cài đặt sẵn nhiều thư viện Python phổ biến cho khoa học dữ liệu và học máy như TensorFlow, PyTorch, NumPy, Pandas và Matplotlib, giúp giảm thêm thời gian thiết lập. Người dùng có thể dễ dàng nhập và làm việc với các bộ dữ liệu lớn trực tiếp trong môi trường Colab.

Mặc dù cung cấp nhiều lợi ích, Colab miễn phí có những giới hạn nhất định như giới hạn thời gian phiên hoạt động, phân bổ tài nguyên tính toán (GPU/TPU) và băng thông có thể thay đổi tùy thuộc vào mức độ sử dụng và tính sẵn có. Các hoạt động lạm dụng dịch vụ cũng bị hạn chế. Để có tài nguyên đảm bảo và cao hơn, người dùng có thể xem xét các gói trả phí như Colab Pro và Colab Pro+.

1.5.3. Ngrok

Ngrok là một dịch vụ ngược proxy đám mây được thiết kế để đưa các dịch vụ mạng cục bộ (thường chạy trên máy tính của nhà phát triển, sau NAT hoặc tường lửa) ra internet công cộng một cách an toàn thông qua các đường hầm bảo mật. Ngrok hoạt động như một điểm truy cập công cộng cho các ứng dụng nội bộ mà không yêu cầu cấu hình mạng phức tạp trên môi trường cục bộ.



Hình 1.5. Ngrok

Điểm mạnh của Ngrok nằm ở sự đơn giản và khả năng hoạt động độc lập với môi trường. Người dùng chỉ cần tải xuống và chạy một chương trình nhỏ trên máy tính chứa dịch vụ cục bộ của mình. Ngrok sẽ tạo ra một địa chỉ công khai trên internet và chuyển tiếp lưu lượng truy cập từ địa chỉ này đến dịch vụ cục bộ của bạn thông qua một đường hầm được mã hóa.

CHƯƠNG 2

PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1. Phân tích hệ thống

2.1.1. Giới thiệu chung

Nhận dạng hóa đơn là quá trình sử dụng phần mềm để quét và chuyển đổi các thông tin trên hóa đơn giấy thành các ký tự có thể đọc được trên máy tính. Khi một hóa đơn được quét, phần mềm OCR sẽ tự động nhận dạng các ký tự trên hóa đơn và chuyển đổi chúng thành văn bản số để lưu trữ hoặc xử lý.

Việc sử dụng OCR hóa đơn giúp cho việc xử lý hóa đơn trở nên nhanh chóng và tiết kiệm thời gian. Nó cũng giảm thiểu sự phụ thuộc vào công việc thủ công và giảm thiểu sai sót do sai sót của con người trong quá trình nhập dữ liệu. Ngoài ra, việc sử dụng OCR còn giúp tăng hiệu suất và độ chính xác của các quy trình kế toán và tài chính.

2.1.2. Yêu cầu chức năng

Hệ thống bao gồm những chức năng sau:

- Gửi yêu cầu: Người dùng có thể gửi yêu cầu xử lý hình ảnh lên hệ thống.
- Chọn hình ảnh: Người dùng có thể chọn ảnh từ thiết bị thay vì dán liên kết hình ảnh.
- Sửa dữ liệu: Người dùng có thể sửa dữ liệu trả về trực tiếp trong bảng hiển thị nếu dữ liệu bị sai.
- Xuất file Excel: Người dùng có thể xuất dữ liệu ra một file excel với mục đích lưu trữ dữ liệu.

2.1.3. Yêu cầu phi chức năng

Giao diện người dùng:

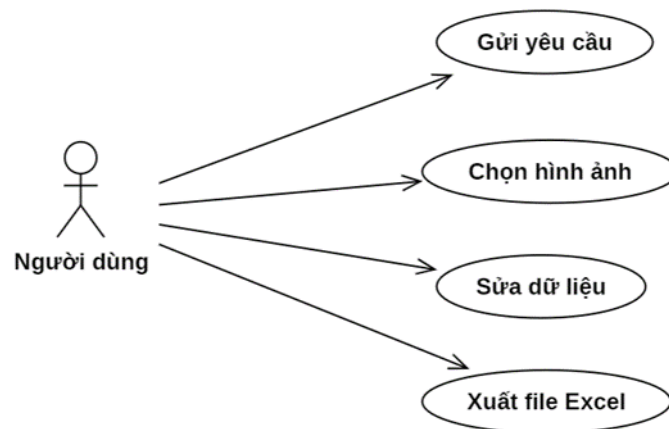
- Giao diện phải được thiết kế đơn giản, dễ sử dụng và thân thiện với người dùng.
- Các nút chức năng cần được bố trí rõ ràng và dễ tìm kiếm.

Tính bảo mật và chính xác:

- Hệ thống phải đảm bảo tính bảo mật của thông tin khách hàng và đơn hàng.
- Dữ liệu được trích xuất với sai sót không có hoặc tối thiểu.
- Phải tuân thủ các quy định pháp luật liên quan đến bảo mật thông tin cá nhân và giao dịch thương mại.

2.2. Đặc tả hệ thống

2.2.1. Biểu đồ Usecase

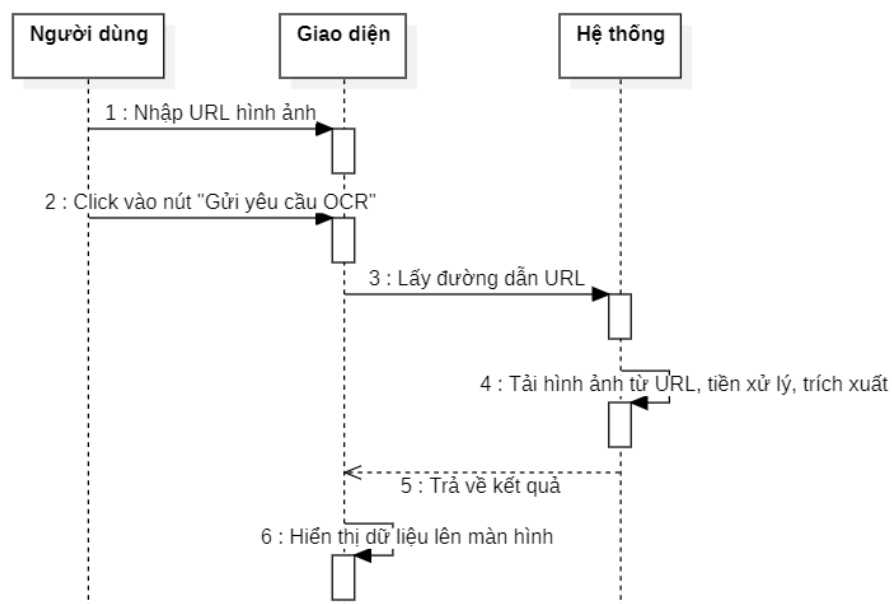


Hình 2.1. Biểu đồ Usecase

2.2.2. Use case *Gửi yêu cầu*

Mô tả	Use case này cho phép người dùng gửi yêu cầu xử lý hình ảnh lên hệ thống.
Luồng sự kiện chính	1. Use case này bắt đầu khi người dùng click vào nút “Gửi yêu cầu OCR” ở màn hình chính. Hệ thống sẽ gửi yêu cầu POST lên Server và hiển thị hướng dẫn Đang xử lý. 2. Hệ thống sẽ trả về kết quả. Use case kết thúc.
Luồng rẽ nhánh	1. Tại bước 1 trong luồng cơ bản, nếu người dùng để trống trường Nhập đường dẫn ảnh, hệ thống sẽ đưa ra thông báo “Vui lòng nhập đường dẫn ảnh.” và yêu cầu nhập đầy đủ thông tin. Use case kết thúc. 2. Tại bất kỳ bước nào trong luồng cơ bản, nếu không kết nối được với Server thì hệ thống sẽ trả về thông báo lỗi và use case kết thúc.
Tiền điều kiện	Không có
Hậu điều kiện	Không có

- Biểu đồ trình tự

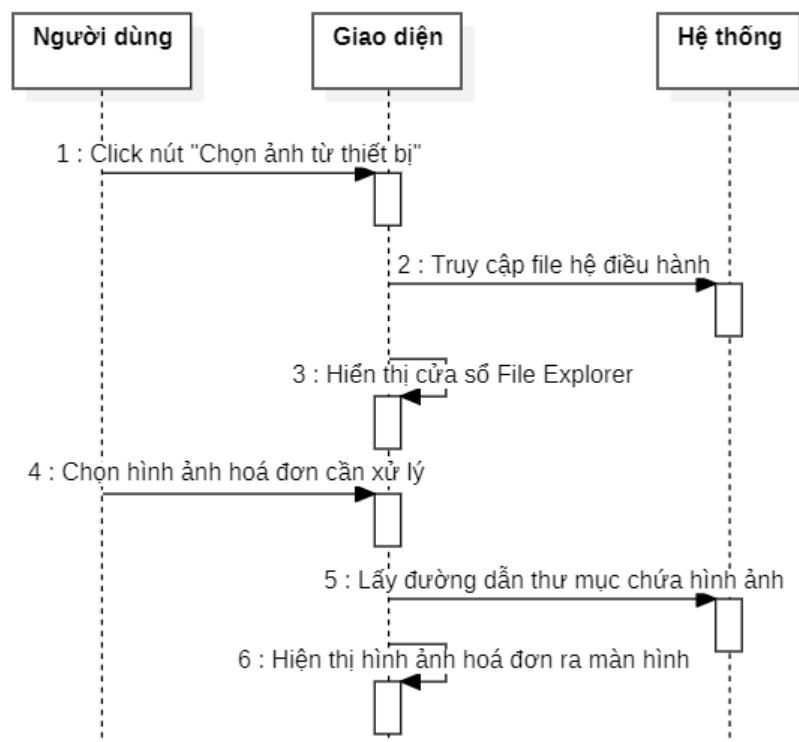


Hình 2.2. Biểu đồ trình tự Usecase *Gửi yêu cầu*

2.2.3. Use case Chọn hình ảnh

Mô tả	Use case này cho phép người dùng chọn ảnh từ thiết bị thay vì liên kết hình ảnh.
Luồng sự kiện chính	1. Use case này bắt đầu khi người dùng click vào nút “Chọn ảnh từ thiết bị” ở màn hình chính. Hệ thống sẽ hiển thị một cửa sổ đến File Explorer, người dùng chọn hình ảnh hoá đơn cần xử lý, 2. Hệ thống sẽ hiển thị ảnh xem trước. Use case kết thúc.
Luồng rẽ nhánh	Tại bước 2 trong luồng cơ bản, nếu người dùng chọn một hình ảnh không phải là hoá đơn, sau khi thực hiện yêu cầu xử lý sẽ trả về thông báo lỗi “Không tìm thấy bảng hợp lệ để hiển thị ở bảng”. Use case kết thúc.
Tiền điều kiện	Không có
Hậu điều kiện	Không có

- Biểu đồ trình tự

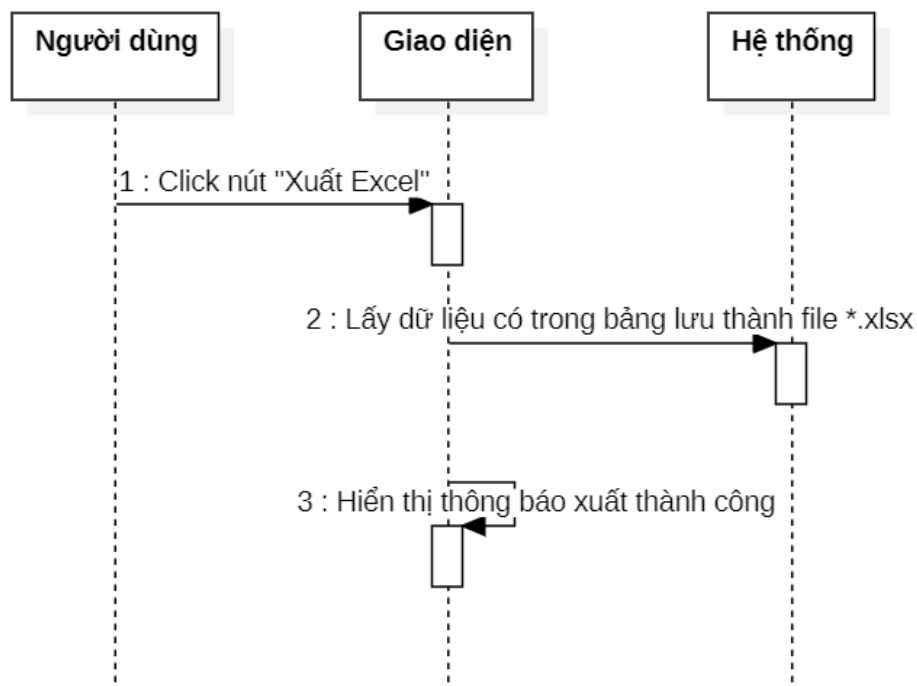


Hình 2.3. Biểu đồ trình tự Usecase Chọn hình ảnh

2.2.4. Use case Xuất file Excel

Mô tả	Use case này cho phép người dùng xuất dữ liệu ra một file excel với mục đích lưu trữ dữ liệu.
Luồng sự kiện chính	1. Use case này bắt đầu khi người dùng click vào nút “Xuất Excel” ở màn hình chính. Hệ thống sẽ lấy dữ liệu từ bảng và xuất ra file với định dạng .xlsx. 2. Hệ thống hiển thị cửa sổ thông báo “Xuất thành công” kèm tên tệp đã lưu. Use case kết thúc.
Luồng rẽ nhánh	1. Tại bước 1 trong luồng cơ bản, nếu trong bảng không có dữ liệu nào hệ thống sẽ trả về thông báo lỗi “Không có bảng nào để xuất”. Use case kết thúc.
Tiền điều kiện	Không có
Hậu điều kiện	Không có

- Biểu đồ trình tự

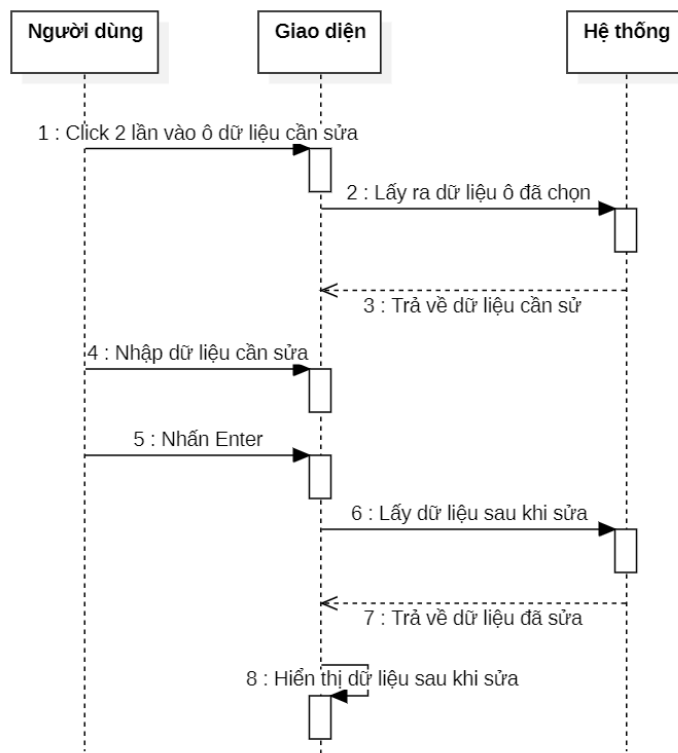


Hình 2.5. Biểu đồ trình tự Usecase Xuất file Excel

2.2.5. Use case Sửa dữ liệu

Mô tả	Use case này cho phép người dùng sửa dữ liệu trả về trực tiếp trong bảng hiển thị nếu dữ liệu bị sai.
Luồng sự kiện chính	1. Use case này bắt đầu khi người dùng click 2 vào ô dữ liệu bị sai ở bảng hiển thị. Hệ thống sẽ cho phép người dùng chỉnh sửa dữ liệu tại ô đã chọn. 2. Người dùng nhấn Enter trên bàn phím sau khi nhập dữ liệu, hệ thống sẽ hiển thị dữ liệu sau khi chỉnh sửa. Use case kết thúc.
Luồng rẽ nhánh	Tại bước 2 trong luồng cơ bản, nếu người dùng không nhấn Enter sau khi chỉnh sửa, dữ liệu sẽ không được lưu. Use case kết thúc.
Tiền điều kiện	Không có
Hậu điều kiện	Không có

- Biểu đồ trình tự



Hình 2.4. Biểu đồ trình tự Usecase Sửa dữ liệu

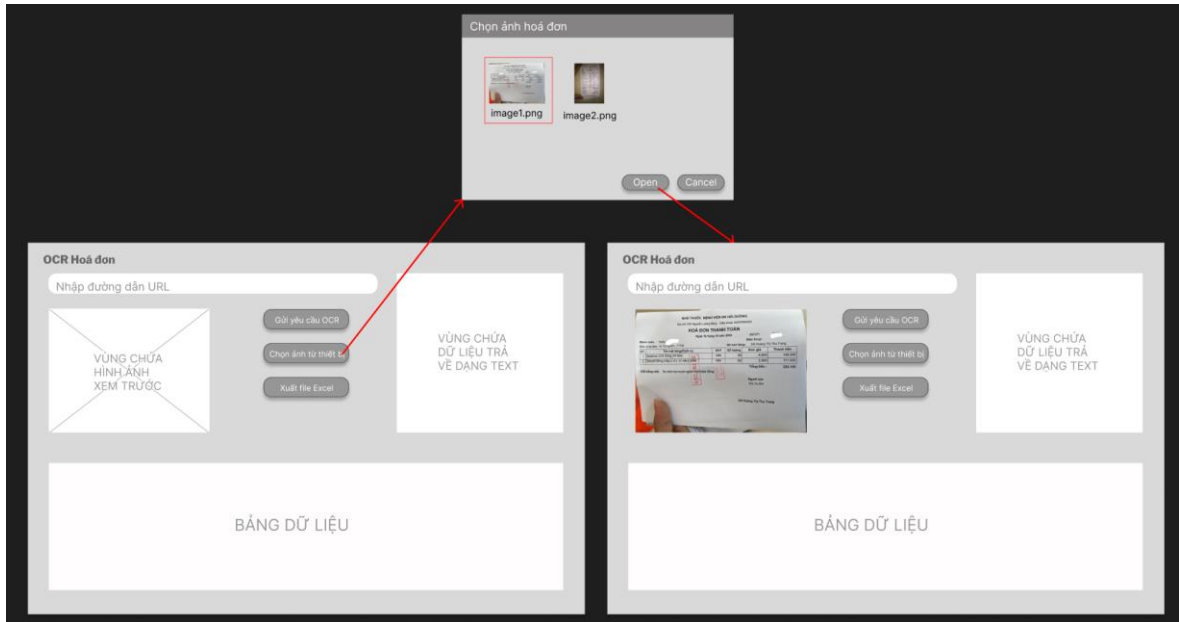
2.3. Thiết kế giao diện

The wireframe illustrates the layout of the OCR application. At the top left, a header 'OCR Hoá đơn' is present. Below it is a text input field labeled 'Nhập đường dẫn URL'. To the right of this field is a large white box labeled 'VÙNG CHỨA DỮ LIỆU TRẢ VỀ DẠNG TEXT'. Below the input field are three buttons: 'Gửi yêu cầu OCR', 'Chọn ảnh từ thiết bị', and 'Xuất file Excel'. To the left of these buttons is a box labeled 'VÙNG CHỨA HÌNH ẢNH XEM TRƯỚC'. At the bottom of the interface is a large white box labeled 'BẢNG DỮ LIỆU'.

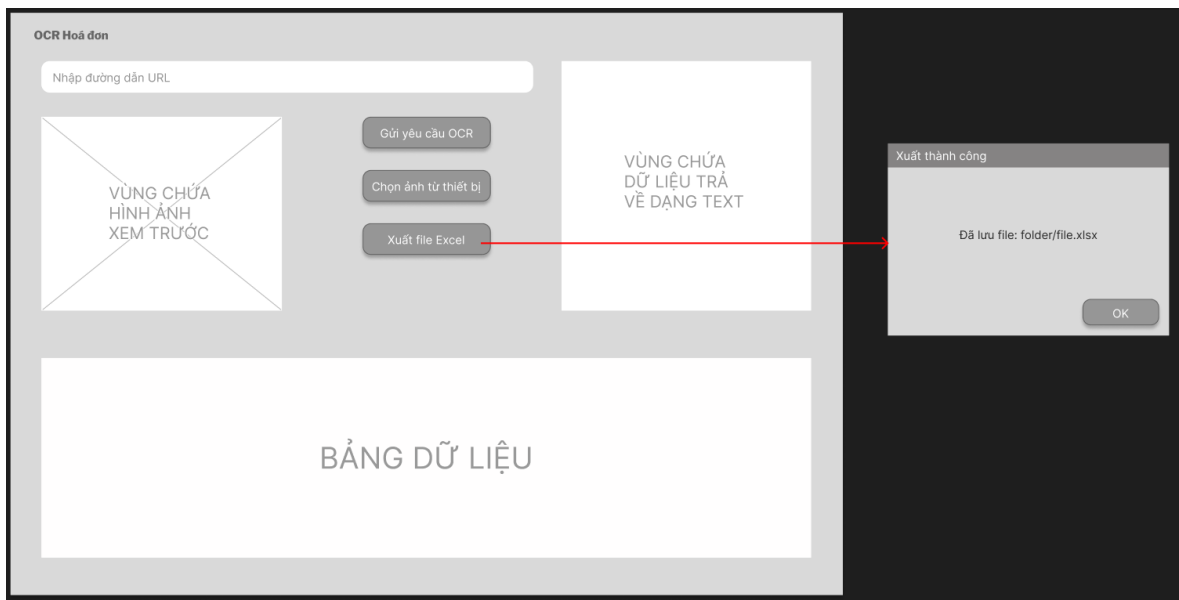
Hình 2.6. Thiết kế giao diện màn hình chính

This wireframe shows the main interface with an additional modal dialog. The main interface is identical to Figure 2.6. A red arrow points from the 'Gửi yêu cầu OCR' button to a modal dialog titled 'Thiếu dữ liệu'. The dialog contains the text 'Vui lòng nhập đường dẫn ảnh' and an 'OK' button.

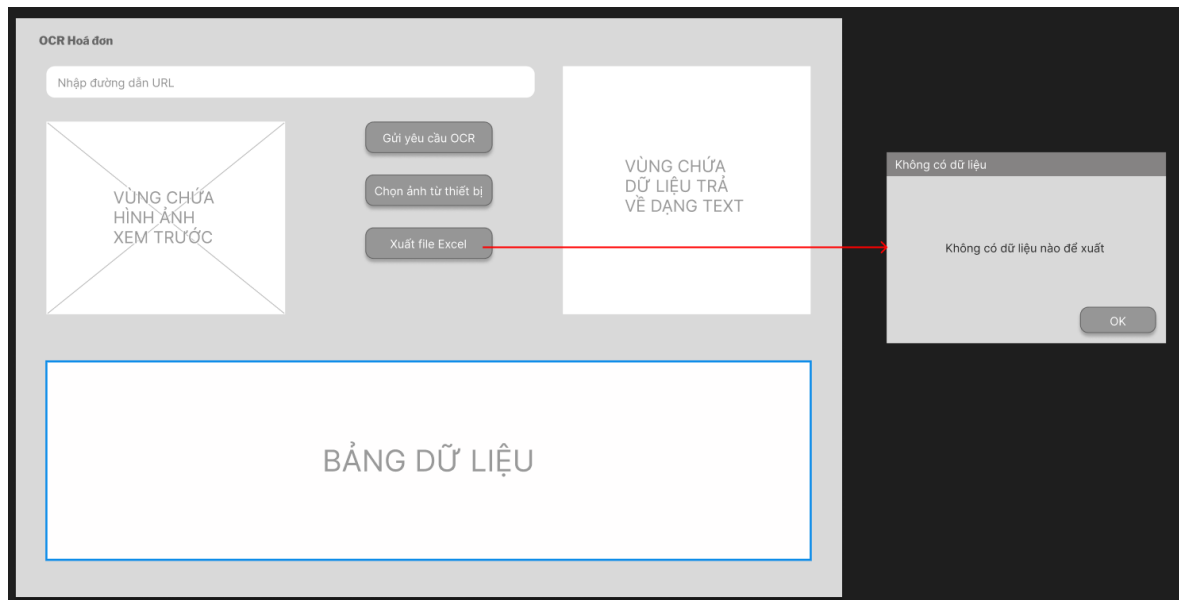
Hình 2.7. Thiết kế giao diện thông báo Nhập đường dẫn ảnh



Hình 2.8. Thiết kế giao diện thông báo Chọn ảnh từ thiết bị



Hình 2.9. Thiết kế giao diện thông báo Xuất file thành công

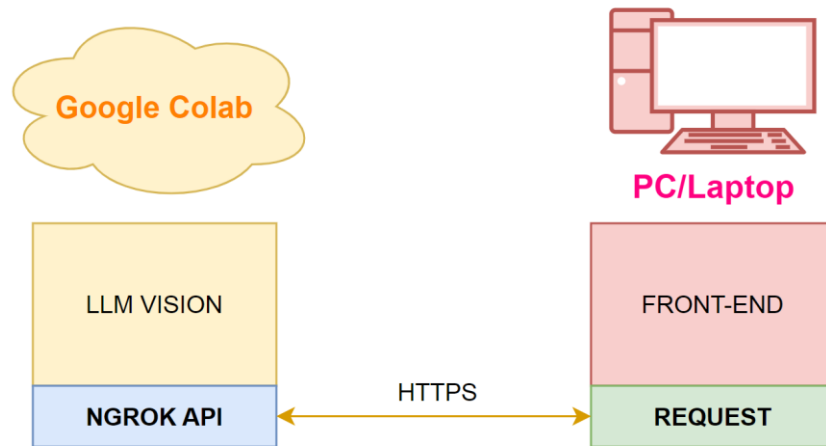


Hình 2.10. Thiết kế giao diện thông báo Xuất file không thành công

CHƯƠNG 3

CÀI ĐẶT VÀ KIỂM THỬ

3.1. Cài đặt và chạy chương trình



Hình 3.1. Nguyên lý hoạt động của chương trình

Trong đó:

- Ứng dụng FRONT-END chạy trên PC/Laptop gửi một REQUEST (yêu cầu) qua giao thức HTTPS.
- Yêu cầu này được chuyển đến địa chỉ công khai được cung cấp bởi NGROK API.
- NGROK API chuyển tiếp yêu cầu này đến ứng dụng LLM VISION đang chạy trên Google Colab.
- LLM VISION xử lý yêu cầu và trả về một phản hồi. Phản hồi này sau đó sẽ đi ngược lại qua Ngrok API và đến ứng dụng Front-End.

3.1.1. Triển khai backend LLM Vision trên Google Colab

Bước 1: Cài đặt và import các thư viện cần thiết

- ``transformers==4.44.2``: Cài đặt thư viện Transformers của Hugging Face (phiên bản 4.44.2) để làm việc với các mô hình ngôn ngữ.
- ``bitsandbytes``: Thư viện này thường được sử dụng để tối ưu hóa việc sử dụng bộ nhớ khi làm việc với các mô hình lớn.

- ``huggingface_hub``: Thư viện để tương tác với Hugging Face Hub, nơi lưu trữ các mô hình và tài nguyên.
- ``flask``, ``flask-cors``: Flask là một framework web micro của Python, và flask-cors hỗ trợ Cross-Origin Resource Sharing (CORS) để cho phép các yêu cầu HTTP từ các nguồn khác nhau.
- ``pyngrok``: Pyngrok được sử dụng để tạo một đường hầm đến localhost, cho phép truy cập từ xa vào các ứng dụng web đang chạy trên Colab.
- ``flash_attn``: Có thể là một thư viện để tăng tốc attention mechanism, một phần quan trọng của transformer models.

Bước 2: Xử lý ảnh đầu vào

Định nghĩa các hàm để tiền xử lý ảnh đầu vào cho mô hình. (Theo **Phụ lục**

3.2. Hàm xử lý ảnh đầu vào)

- `build_transform`: Tạo một pipeline biến đổi ảnh (resize, chuyển đổi sang RGB, chuyển sang tensor, chuẩn hóa).
- `dynamic_preprocess`: Một hàm phức tạp để xử lý ảnh đầu vào một cách linh hoạt, chia ảnh thành các phần nhỏ hơn dựa trên tỷ lệ khung hình và kích thước mong muốn. Điều này có thể giúp mô hình xử lý ảnh có độ phân giải khác nhau một cách hiệu quả.
- `load_image`: Hàm này tải ảnh từ một URL, tiền xử lý nó bằng các hàm đã định nghĩa, và trả về một tensor sẵn sàng cho mô hình.

Bước 3: Load và test model

- ``model_name = "5CD-AI/Vintern-1B-v2"``: Chỉ định tên của mô hình được tải từ Hugging Face Hub.
- ``AutoModel.from_pretrained(...)``: Tải mô hình. Các tham số quan trọng:
 - ``torch_dtype=torch.bfloat16``: Sử dụng kiểu dữ liệu bfloat16 để giảm mức sử dụng bộ nhớ và tăng tốc tính toán.

- ``low_cpu_mem_usage=True``: Cố gắng giảm mức sử dụng bộ nhớ CPU.
 - ``trust_remote_code=True``: Cho phép thực thi mã từ mô hình (cần thận trọng khi sử dụng).
 - ``eval().cuda()``: Chuyển mô hình sang chế độ đánh giá và đưa nó lên GPU.
- ``tokenizer = AutoTokenizer.from_pretrained(...)``: Tải tokenizer tương ứng với mô hình.
 - ``generation_config``: Cấu hình các tham số cho quá trình sinh văn bản của mô hình.
 - Tải một ảnh từ URL, tiền xử lý nó, và sử dụng mô hình để tạo ra một mô tả hoặc phân tích về ảnh. Đoạn prompt yêu nhận diện hoá đơn trong ảnh và trả về thông tin dưới dạng JSON.
 - Kết quả:

```
'[\n {\n   "Tên món": "Giờ VIP222",\n   "Số lượng": "1h54'\n   "Đơn giá": "500 000",\n   "Thành tiền": "950 000"\n },\n {\n   "Tên món": "Suối",\n   "Số lượng": "3",\n   "Đơn giá": "12 000",\n   "Thành tiền": "36 000"\n },\n {\n   "Tên món": "Hoa quả thập cẩm",\n   "Số lượng": "1",\n   "Đơn giá": "140 000",\n   "Thành tiền": "140 000"\n },\n {\n   "Tên món": "Hoa quả Bưởi",\n   "Số lượng": "2",\n   "Đơn giá": "220 000",\n   "Thành tiền": "440 000"\n },\n {\n   "Tên món": "Hoa Quả Roi",\n   "Số lượng": "1",\n   "Đơn giá": "100 000",\n   "Thành tiền": "100 000"\n },\n {\n   "Tên món": "Ken ngoại",\n   "Số lượng": "14",\n   "Đơn giá": "60 000",\n   "Thành tiền": "840 000"\n }\n ]'
```

Bước 4: Triển khai Flask và Expose ra API qua Ngrok

(Theo **Phụ lục 3.4. Triển khai Flask và Expose ra API qua Ngrok**)

- Triển khai một API web đơn giản bằng Flask và sử dụng Ngrok để cung cấp quyền truy cập công khai vào API đó.
- Cho phép người dùng gửi yêu cầu HTTP đến Colab notebook và nhận kết quả từ mô hình.
- Kết quả:

```
NgrokTunnel: "https://d383-34-125-151-220.ngrok-free.app" -> "http://localhost:5555"
* Serving Flask app '__main__'
* Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production de
* Running on http://127.0.0.1:5555
INFO:werkzeug:Press CTRL+C to quit
```

3.1.2. Triển khai trường trình phía client trên VS Code

Bước 1: Import các thư viện cần thiết

Bao gồm:

- requests: Để thực hiện các yêu cầu HTTP đến API.
- tkinter: Để tạo giao diện người dùng đồ họa (GUI).
- PIL (Pillow) Image, ImageTk: Để xử lý hình ảnh và hiển thị hình ảnh trong Tkinter.
- os: Để tương tác với đường dẫn tệp, thao tác thư mục.
- datetime: Để làm việc với ngày và giờ.
- base64: Để mã hóa và giải mã dữ liệu nhị phân.
- openpyxl: Để đọc và ghi các tệp Excel (.xlsx).

Bước 2: Xây dựng hàm giao tiếp với API OCR

Bao gồm:

- Tạo một dictionary payload để chứa dữ liệu gửi đến API.
- Kiểm tra xem image_url hay image_base64 được cung cấp và thêm khóa tương ứng vào payload.
- Nếu không có hình ảnh nào được cung cấp, trả về thông báo lỗi.
- Sử dụng thư viện requests để gửi một yêu cầu POST đến địa chỉ API. Dữ liệu payload được gửi dưới dạng JSON.
- Xử lý phản hồi từ API:
 - Nếu mã trạng thái HTTP là 200 (thành công), hàm sẽ cố gắng phân tích JSON trong phản hồi và trả về giá trị của khóa "response_message" (kết quả OCR dạng bảng) và "raw_output" (đầu ra thô). Nếu các khóa này không tồn tại, nó sẽ trả về None cho giá trị tương ứng.
 - Nếu mã trạng thái khác 200, hàm trả về một thông báo lỗi chứa mã trạng thái và nội dung lỗi từ API.

- Xử lý các ngoại lệ có thể xảy ra trong quá trình gửi yêu cầu (ví dụ: lỗi mạng) và trả về thông báo lỗi.

Bước 3: Xây dựng hàm gửi yêu cầu

Bao gồm:

- Lấy đường dẫn URL từ ô nhập `url_entry` và loại bỏ khoảng trắng thừa.
- Kiểm tra xem URL có được nhập hay không. Nếu không, hiển thị cảnh báo.
- Hiển thị thông báo "Đang xử lý..." bằng cách cấu hình `loading_label`.
- Gọi hàm `perform_ocr()` với URL đã nhập để thực hiện OCR.
- Ẩn thông báo "Đang xử lý..."
- Gọi hàm `handle_result()` để xử lý kết quả OCR và hiển thị nó.

Bước 4: Xây dựng hàm chọn ảnh từ thiết bị

Bao gồm:

- Mở hộp thoại chọn tệp để người dùng chọn một tệp hình ảnh (.png, .jpg, .jpeg, .bmp).
- Nếu người dùng chọn một tệp:
 - Gọi hàm `display_image_preview()` để hiển thị ảnh đã chọn.
 - Hiển thị thông báo "Đang xử lý..."
 - Gọi hàm `encode_image_base64()` để mã hóa tệp ảnh thành chuỗi Base64.
 - Gọi hàm `perform_ocr()` với chuỗi Base64 để thực hiện OCR.
 - Ẩn thông báo "Đang xử lý..."
 - Gọi hàm `handle_result()` để xử lý kết quả OCR và hiển thị nó.

Bước 5: Hiển thị ảnh xem trước

Bao gồm:

- Sử dụng thư viện PIL (Pillow) để mở hình ảnh.
- Tạo một ảnh thumbnail (kích thước nhỏ hơn) để hiển thị.
- Chuyển đổi ảnh PIL thành đối tượng ImageTk.PhotoImage của Tkinter.
- Cấu hình preview_label để hiển thị ảnh PhotoImage.
- Giữ một tham chiếu đến photo trong thuộc tính preview_label.image để ngăn garbage collector xóa ảnh.
- Nếu có lỗi xảy ra trong quá trình hiển thị ảnh, hiển thị thông báo lỗi.

Bước 6: Xây dựng hàm phân tích một chuỗi văn bản và trích xuất các tiêu đề cột và các hàng dữ liệu cho bảng.

Bao gồm:

- Loại bỏ khoảng trắng thừa ở đầu và cuối chuỗi text.
- Chia chuỗi thành một danh sách các dòng bằng cách sử dụng \n làm dấu phân cách.
- Lặp qua từng dòng và loại bỏ khoảng trắng thừa ở đầu và cuối mỗi dòng. Bỏ qua các dòng trống.
- Kiểm tra xem dòng có bắt đầu và kết thúc bằng dấu | hay không (đây là quy ước cho một dòng dữ liệu bảng).
- Nếu là một dòng dữ liệu bảng, chia dòng thành các ô bằng cách sử dụng | làm dấu phân cách và loại bỏ khoảng trắng thừa ở mỗi ô.
- Nếu danh sách headers (tiêu đề cột) vẫn trống, dòng hiện tại được coi là dòng tiêu đề và được gán cho headers.
- Nếu headers đã có dữ liệu, dòng hiện tại được coi là một hàng dữ liệu và được thêm vào danh sách data_rows.
- Để đảm bảo số lượng cột nhất quán, hàm sẽ thêm các ô trống vào cuối các hàng dữ liệu nếu chúng có ít cột hơn headers, và thêm các tiêu đề cột mặc định ("Cột N") vào headers nếu một hàng dữ liệu có nhiều cột hơn headers.

Bước 7: Xây dựng hàm cho phép sửa dữ liệu trực tiếp trong bảng nếu bị sai

Bao gồm:

- Xác định hàng (row_id) và cột (column_id) mà người dùng đã nhấp vào. Nếu không xác định được, hàm sẽ thoát.
- Lấy tọa độ (x, y, width, height) của ô đã nhấp.
- Lấy chỉ số cột (column_index) từ column_id.
- Lấy giá trị hiện tại của ô đã nhấp.
- Tạo một widget tk.Entry (ô nhập liệu) trên Treeview tại vị trí và kích thước của ô đã nhấp.
- Chèn giá trị hiện tại của ô vào edit_entry và đặt focus vào nó.
- Định nghĩa một hàm nội bộ save_edit(event) để lưu giá trị đã chỉnh sửa khi người dùng nhấn Enter. Hàm này sẽ lấy giá trị mới từ edit_entry, cập nhật giá trị trong Treeview và hủy edit_entry.

Bước 8: Xây dựng hàm cho phép xuất dữ liệu trong bảng thành file Excel

Bao gồm:

- Kiểm tra xem có tiêu đề cột ('current_headers') và dữ liệu trong 'Treeview' hay không. Nếu không, hiển thị cảnh báo.
- Tạo một thư mục có tên '"data_result"' nếu nó chưa tồn tại.
- Tạo một tên tệp Excel duy nhất bằng cách sử dụng dấu thời gian hiện tại.
- Tạo một đối tượng 'Workbook' mới từ thư viện 'openpyxl'.
- Lấy sheet hoạt động ('ws').
- Đặt tiêu đề của sheet thành '"OCR Result"'.
- Thêm các tiêu đề cột ('current_headers') vào hàng đầu tiên của sheet.
- Lặp qua tất cả các hàng trong 'Treeview' và thêm dữ liệu của mỗi hàng vào một hàng mới trong sheet.

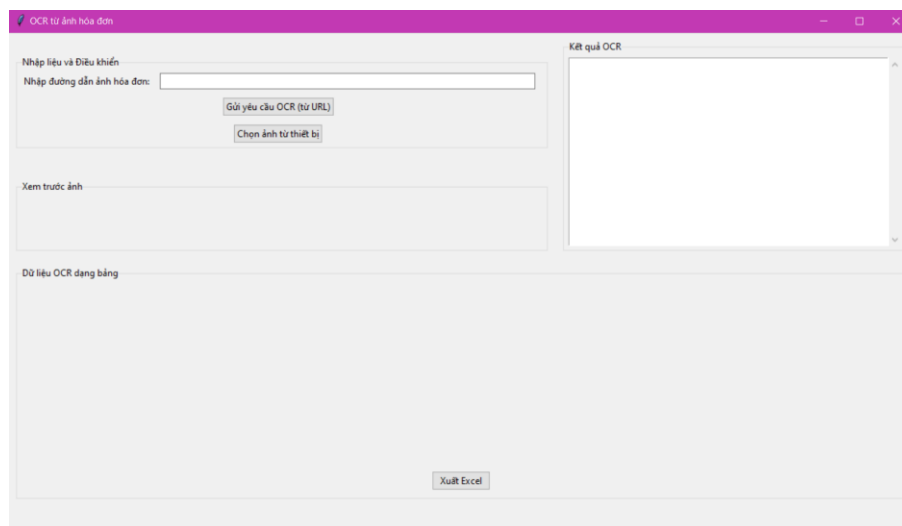
- Lưu workbook vào tệp Excel đã tạo.
- Hiện thị thông báo thành công kèm theo tên tệp đã lưu.

3.1.3. Kết quả

3.1.3.1. Giao diện chính

Bao gồm:

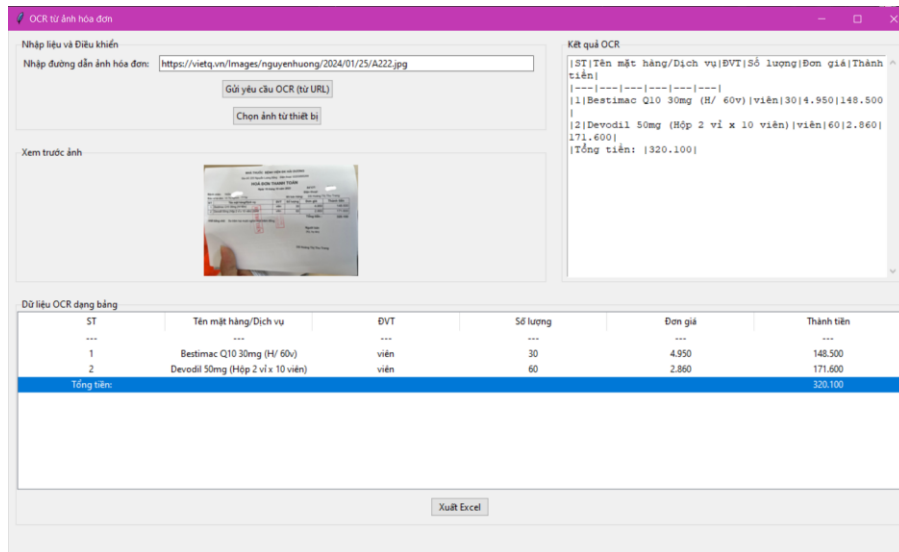
- 1 textfield cho phép người dùng nhập, dán đường dẫn hình ảnh
- 1 nút “Gửi yêu cầu OCR” cho phép người dùng gửi yêu cầu xử lý hình ảnh lên server để xử lý.
- 1 nút “Chọn ảnh từ thiết bị” cho phép người dùng chọn ảnh từ thiết bị thay vì nhập, dán đường dẫn hình ảnh.
- 1 vùng chứa hình ảnh xem trước.
- 1 scrolltext để hiển thị dữ liệu thô chưa xử lý, thông báo lỗi từ server.
- 1 bảng chứa dữ liệu: hiển thị dữ liệu trả về dưới dạng bảng
- 1 nút “Xuất Excel” cho phép người dùng xuất dữ liệu từ bảng ra file Excel định dạng .xls.



Hình 3.1. Giao diện chính

3.1.3.2. Giao diện hiển thị kết quả

Hiển thị dữ liệu sau khi người dùng đã gửi yêu cầu và nhận lại kết quả từ server:



Nhập liệu và Điều khiển

Nhập đường dẫn ảnh hóa đơn:

Xem trước ảnh

Kết quả OCR

```

[ST|Tên mặt hàng/Dịch vụ|ĐVT|Số lượng|Đơn giá|Thành
tiền]
|---|---|---|---|---|
|1|Bestimac Q10 30mg (H/ 60v)|viên|30|4.950|148.500
|
|2|Devodil 50mg (Hộp 2 vỉ x 10 viên)|viên|60|2.860|
171.600|
|Tổng tiền: |320.100|
  
```

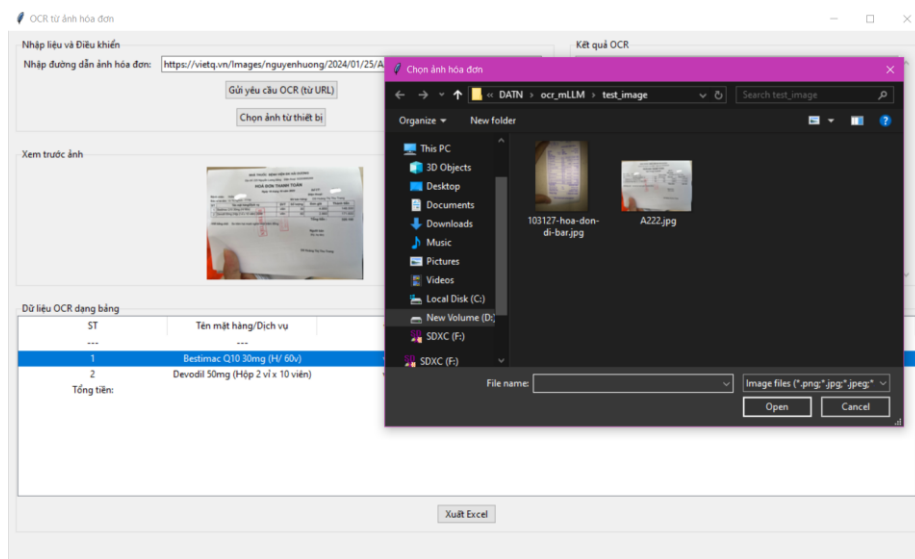
Dữ liệu OCR dạng bảng

ST	Tên mặt hàng/Dịch vụ	ĐVT	Số lượng	Đơn giá	Thành tiền
1	Bestimac Q10 30mg (H/ 60v)	viên	30	4.950	148.500
2	Devodil 50mg (Hộp 2 vỉ x 10 viên)	viên	60	2.860	171.600
Tổng tiền:					320.100

Hình 3.2. Giao diện hiển thị kết quả

3.1.3.3. Cửa sổ chọn hình ảnh

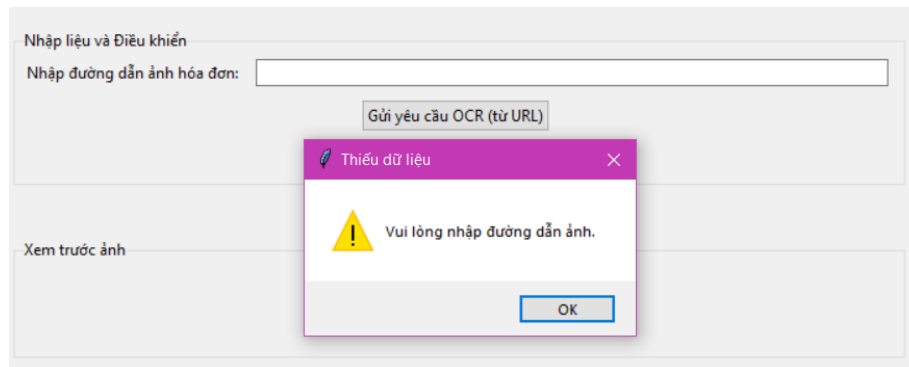
- Khi người dùng bấm nút “Chọn ảnh từ thiết bị“, cửa sổ File Explorer hiện lên, trở đến folder chứa ảnh hoá đơn, khi người dùng chọn xong sẽ hiển thị ảnh xem trước của hoá đơn vừa chọn:



Hình 3.3. Cửa sổ chọn hình ảnh từ thiết bị

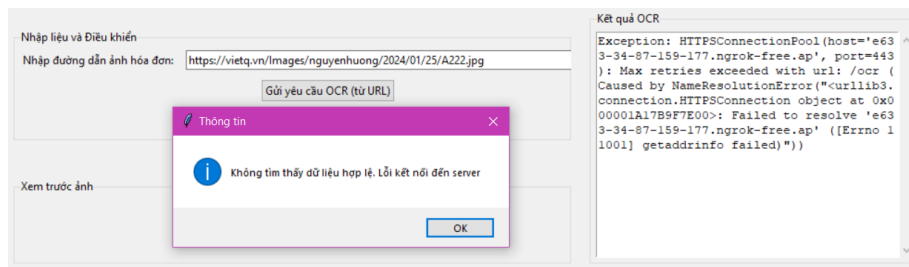
3.13.4. Các cửa sổ thông báo

- Cửa sổ thông báo khi người dùng để đường dẫn ảnh trống:



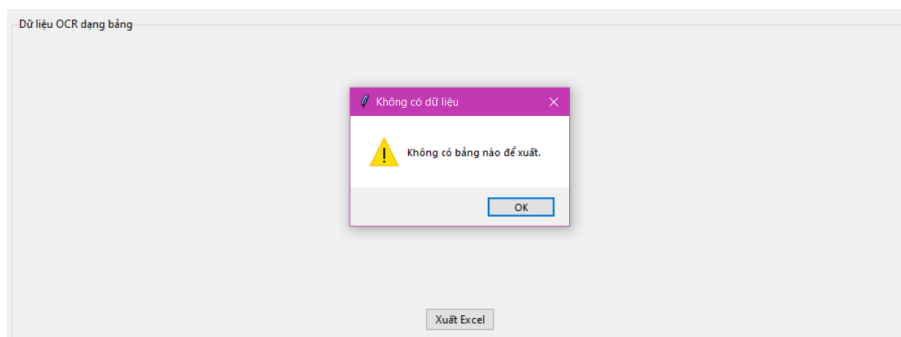
Hình 3.4. Cửa sổ thông báo khi để đường dẫn ảnh trống

- Cửa sổ thông báo khi người dùng gặp lỗi kết nối server:



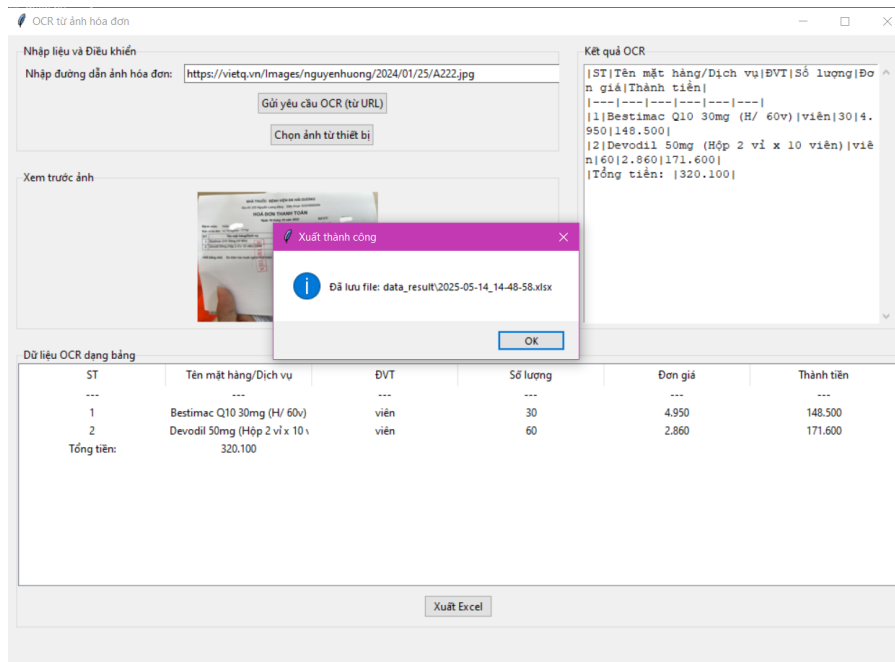
Hình 3.5. Cửa sổ thông báo lỗi kết nối server

- Cửa sổ thông báo khi người dùng bấm nút Xuất Excel nhưng trong bảng không hiển thị dữ liệu nào:



Hình 3.6. Cửa sổ thông báo nếu dữ liệu trong bảng trống

- Cửa sổ thông báo khi người dùng bấm nút Xuất Excel và dữ liệu được xuất thành công:



Hình 3.7. Cửa sổ thông báo xuất file thành công

3.2. Kiểm thử

3.2.1. Chuẩn bị dữ liệu kiểm thử

- Hóa đơn thực tế đa dạng: Thu thập một bộ sưu tập lớn các hóa đơn thực tế từ nhiều nguồn, nhiều nhà cung cấp, với các bố cục, phong chữ, chất lượng hình ảnh khác nhau.
- Các trường hợp biên: Bao gồm các hóa đơn có thông tin khó đọc, bị cắt, bị mờ, có chữ viết tay, hoặc có các định dạng đặc biệt.

3.2.2. Mục tiêu kiểm thử

- Đánh giá độ chính xác của việc trích xuất thông tin từ hóa đơn bán lẻ.
- Kiểm tra khả năng xử lý các hóa đơn có chất lượng hình ảnh kém.
- Đánh giá hiệu suất xử lý dưới tải.

3.2.3. Phạm vi kiểm thử

Trích xuất thông tin cơ bản (tên công ty, ngày, tổng tiền), trích xuất mục hàng, xử lý ảnh nghiêng/thiếu sáng.

3.2.4. Phương pháp kiểm thử

- Kiểm thử thủ công và tự động.
- Kiểm thử hiệu suất.

3.2.5. Kết quả kiểm thử

- Độ chính xác trích xuất

Trường thông tin	Precision (%)	Recall (%)	F1-score (%)
Tên công ty	95.2	93.8	94.5
Ngày hóa đơn	98.1	97.5	97.8
Tổng tiền	96.5	95	95.7
Mục hàng	88	85.5	86.7

Bảng 3.1. Tỷ lệ độ chính xác trích xuất

- Bảng kết quả lỗi ký tự (CER), lỗi từ (WER) theo chất lượng ảnh:

Loại hình ảnh	CER (%)	WER (%)
Chất lượng tốt	1.5	3.2
Nghiêng	4.8	9.1
Thiếu sáng	6.2	11.5
Mờ	7.1	13

Bảng 3.2. Tỷ lệ lỗi ký tự, lỗi từ

- Lỗi cơ bản:

Loại	Kết quả mong đợi	Kết quả thực tế	Phân tích lỗi
Hóa đơn siêu thị bị chụp mờ	Tổng cộng:150 000	Tổng cộng:15 000	Phạm diện sai dấu phân cách hàng nghìn, dẫn đến sai lệch giá trị tổng tiền. Nguyên nhân có thể do hình ảnh quá mờ, khiến ký tự

			"0" bị nhầm lẫn với khoảng trắng hoặc dấu chấm.
Hóa đơn viết tay, chất lượng kém	Tên sản phẩm: Bút bi Thiên Long	Tên sản phẩm: Bút bi Thien Long	Chữ viết tay không chuẩn hoặc quá phức tạp, mô hình văn gặp khó khăn trong việc nhận diện chính xác từng ký tự, đặc biệt là các từ tiếng Việt có dấu.
Hóa đơn dịch vụ với nhiều bảng biểu	Mã khách hàng: KH12345	Mã số: KH12345	Trích xuất đúng giá trị nhưng sai tên trường (key). Điều này cho thấy mô hình có thể gặp khó khăn trong việc hiểu ngữ cảnh và phân loại đúng các trường thông tin

Bảng 3.3. Phân tích một số lỗi cơ bản

- Khả năng chịu lỗi và biến thể:
 - Có khả năng xử lý tương đối tốt các hóa đơn bị nghiêng nhẹ (dưới 20 độ) và thiếu sáng vừa phải, với độ chính xác giảm khoảng 5-10% so với hóa đơn chuẩn. Tuy nhiên, khi độ nghiêng vượt quá 30 độ hoặc hình ảnh quá tối/quá sáng, độ chính xác giảm mạnh (lên đến 20-30% cho các trường quan trọng).
 - Mô hình gặp nhiều thách thức hơn với hóa đơn bị nhòe, rách hoặc có chữ viết tay không rõ ràng.
- Hiệu suất:
 - Thời gian xử lý trung bình một hóa đơn (kích thước ~1MB) trên môi trường kiểm thử là 2.5 giây.
 - Khi xử lý 10 hóa đơn đồng thời, thời gian trung bình tăng lên 3.2 giây/hóa đơn, cho thấy khả năng mở rộng tốt ở mức tải vừa phải.

- Mức tiêu thụ GPU VRAM trung bình là 6GB cho mỗi phiên xử lý.

- Chức năng:

Tên test case	Số thứ tự các bước	Tên các bước	Kết quả mong đợi	Kết quả Thực tế	Trạng thái
Gửi yêu cầu	1	Nhập đầy đủ trường URL hình ảnh	Trả về đầy đủ dữ liệu	Trả về đầy đủ dữ liệu	Thành công
	2	Không nhập đường dẫn hình ảnh	Đưa ra thông báo “Vui lòng nhập đường dẫn ảnh.”	Đưa ra thông báo “Vui lòng nhập đường dẫn ảnh.”	Thành công
	3	Gửi ảnh từ thiết bị	Trả về đầy đủ dữ liệu	Lỗi 500	Không thành công
Chọn hình ảnh	1	Chọn hình ảnh từ thiết bị	Hiển thị ảnh kèm đường dẫn trên giao diện	Hiển thị ảnh kèm đường dẫn trên giao diện	Thành công
Sửa dữ liệu	1	Sửa dữ liệu tại ô dữ liệu bị sai	Dữ liệu sau khi sửa được cập nhật vào bảng	Dữ liệu sau khi sửa được cập nhật vào bảng	Thành công
Xuất file Excel	1	Xuất file excel với bảng trống	Đưa ra thông báo “Không có bảng nào để xuất”	Đưa ra thông báo “Không có bảng nào để xuất”	Thành công

	2	Xuất file excel với bảng đầy đủ dữ liệu	Dữ liệu xuất ra đầy đủ, đưa ra thông báo “Xuất thành công”	Dữ liệu xuất ra đầy đủ, đưa ra thông báo “Xuất thành công”	Thành công
	3	Xuất file excel với bảng dữ liệu sau khi đã sửa	Dữ liệu xuất ra đầy đủ, đưa ra thông báo “Xuất thành công”	Dữ liệu xuất ra đầy đủ, đưa ra thông báo “Xuất thành công”	Thành công

Bảng 3.4. Kiểm thử các chức năng

3.2.6. Kết luận

Ứng dụng OCR hóa đơn sử dụng Vintern-1B-v2 đã đạt được các mục tiêu kiểm thử cơ bản về khả năng trích xuất thông tin từ hóa đơn tiếng Việt. Mặc dù có hiệu suất tốt trên các hóa đơn chất lượng cao, vẫn còn những thách thức đáng kể với các trường hợp biên và yêu cầu cải thiện về trải nghiệm người dùng. Với các khuyến nghị đã nêu, ứng dụng có tiềm năng lớn để trở thành một công cụ mạnh mẽ và đáng tin cậy hơn trong việc tự động hóa quy trình xử lý hóa đơn.

KẾT LUẬN

4.1. Kiến thức đạt được

- Nắm chắc các kiến thức căn bản của đề tài thực hiện
- Sử dụng thành thạo ngôn ngữ, công cụ trong quá trình phát triển đề tài.
- Hiểu biết thêm cách sử dụng ngôn ngữ và công cụ liên quan tới đề tài nghiên cứu.
- Rút ra được kinh nghiệm, bài học cần có trong quá trình phát triển ứng dụng.

4.2. Kỹ năng đạt được

- Kỹ năng đọc, hiểu.
- Phát triển và rèn luyện các kỹ năng tư duy sáng tạo, làm việc độc lập.
- Phát triển và rèn luyện kỹ năng mềm cần có để thực hiện đề tài.
- Ứng dụng thành thạo ngôn ngữ Python vào trong quá trình phát triển phần mềm.

4.3. Bài học kinh nghiệm

Linh hoạt và sẵn sàng trong mọi tình huống:

- Khi hoàn thành đề tài bạn lại nghĩ ra được phương án tốt hơn, tối ưu hơn và thay thế chương trình bạn vừa hoàn thành kia là một điều hết sức bình thường. Lập trình thay đổi rất nhanh và ngày càng tối ưu hơn.

- Cùng một vấn đề nhưng có nhiều cách tiếp cận nhanh hơn, tốt hơn và tìm được giải pháp tối ưu hơn mới là lý do của lập trình. Vì thế với một người học lập trình thì linh hoạt và có thể ứng phó trong mọi tình huống là điều quan trọng mà người lập trình cần phải chú tâm.

Kiên định và không từ bỏ: Cố gắng thay đổi bản thân để thích nghi và thử nghiệm với những điều mới. Mặc dù khó nhưng càng tiếp xúc, kiên trì bền bỉ thì tư duy lập trình sẽ được mở rộng rất nhiều.

TÀI LIỆU THAM KHẢO

- [1] Rabiloo. (2024). Tổng quan về mô hình thác nước (Waterfall Model): <https://rabiloo.com.vn/blog/tong-quan-ve-mo-hinh-thac-nuoc-waterfall-model>.
- [2] VTC. (n.d.). Kiến trúc phần mềm là gì: <https://plus.vtc.edu.vn/kien-truc-phan-mem-la-gi>.
- [3] Doan, K. T., et al. (2024). Vintern-1B: An Efficient Multimodal Large Language Model for Vietnamese. *arXiv preprint arXiv:2408.12480*.
- [4] 5CD-AI. (n.d.). Vintern-1B-v2: <https://huggingface.co/5CD-AI/Vintern-1B-v2>.
- [5] Dai, J., Chen, Z., Wang, Y., Liu, Y., Gao, Z., Cui, E., ... & Qiao, Y. (2024). InternVL 2.0: Scaling up vision foundation models for versatile visual capabilities. *arXiv preprint arXiv:2403.06650*.
- [6] Hoàng Quang Huy (2016), “*Giáo trình kiểm thử phần mềm*”, Nhà xuất bản Thống kê.

PHỤ LỤC

Phụ lục 3.1. Các thư viện cần thiết

```
!pip install -U -q transformers==4.44.2 bitsandbytes
!pip install -U -q huggingface_hub
!pip install -q flask flask-cors pyngrok flash_attn
import numpy as np
import torch
import torchvision.transforms as T
from PIL import Image
from torchvision.transforms.functional import InterpolationMode
from transformers import AutoModel, AutoTokenizer
import requests
```

Phụ lục 3.2. Hàm xử lý ảnh đầu vào

```
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

def build_transform(input_size):
    MEAN, STD = IMAGENET_MEAN, IMAGENET_STD
    transform = T.Compose([
        T.Lambda(lambda img: img.convert('RGB') if img.mode != 'RGB'
else img),
        T.Resize((input_size, input_size),
interpolation=InterpolationMode.BICUBIC),
        T.ToTensor(),
        T.Normalize(mean=MEAN, std=STD)
    ])
    return transform

def find_closest_aspect_ratio(aspect_ratio, target_ratios, width,
height, image_size):
    best_ratio_diff = float('inf')
    best_ratio = (1, 1)
    area = width * height
    for ratio in target_ratios:
        target_aspect_ratio = ratio[0] / ratio[1]
        ratio_diff = abs(aspect_ratio - target_aspect_ratio)
        if ratio_diff < best_ratio_diff:
            best_ratio_diff = ratio_diff
            best_ratio = ratio
        elif ratio_diff == best_ratio_diff:
            if area > 0.5 * image_size * image_size * ratio[0] *
ratio[1]:
                best_ratio = ratio
    return best_ratio

def dynamic_preprocess(image, min_num=1, max_num=12, image_size=448,
use_thumbnail=False):
```

```

orig_width, orig_height = image.size
aspect_ratio = orig_width / orig_height

# calculate the existing image aspect ratio
target_ratios = set(
    (i, j) for n in range(min_num, max_num + 1) for i in range(1,
n + 1) for j in range(1, n + 1) if
    i * j <= max_num and i * j >= min_num)
target_ratios = sorted(target_ratios, key=lambda x: x[0] * x[1])

# find the closest aspect ratio to the target
target_aspect_ratio = find_closest_aspect_ratio(
    aspect_ratio, target_ratios, orig_width, orig_height,
image_size)

# calculate the target width and height
target_width = image_size * target_aspect_ratio[0]
target_height = image_size * target_aspect_ratio[1]
blocks = target_aspect_ratio[0] * target_aspect_ratio[1]

# resize the image
resized_img = image.resize((target_width, target_height))
processed_images = []
for i in range(blocks):
    box = (
        (i % (target_width // image_size)) * image_size,
        (i // (target_width // image_size)) * image_size,
        ((i % (target_width // image_size)) + 1) * image_size,
        ((i // (target_width // image_size)) + 1) * image_size
    )
    # split the image
    split_img = resized_img.crop(box)
    processed_images.append(split_img)
assert len(processed_images) == blocks
if use_thumbnail and len(processed_images) != 1:
    thumbnail_img = image.resize((image_size, image_size))
    processed_images.append(thumbnail_img)
return processed_images

def load_image(image_file, input_size=448, max_num=12):
    image = Image.open(requests.get(image_file,
stream=True).raw).convert('RGB')
    transform = build_transform(input_size=input_size)
    images = dynamic_preprocess(image, image_size=input_size,
use_thumbnail=True, max_num=max_num)
    pixel_values = [transform(image) for image in images]
    pixel_values = torch.stack(pixel_values)
    return pixel_values

```

Phụ lục 3.3. Load và test model

```

model_name = "5CD-AI/Vintern-1B-v2"
model = AutoModel.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    low_cpu_mem_usage=True,
    trust_remote_code=True,
).eval().cuda()

tokenizer = AutoTokenizer.from_pretrained(model_name,
trust_remote_code=True, use_fast=False)
generation_config = dict(max_new_tokens= 512, do_sample=False,
num_beams = 3, repetition_penalty=3.5)
test_image = 'https://demo.link/image.jpg'

pixel_values = load_image(test_image,
max_num=6).to(torch.bfloat16).cuda()

prompt = '''<image>\nNhận diện hoá đơn trong ảnh. Chỉ trả về phần
liệt kê các mặt hàng hàng dưới dạng JSON:
[
    {
        "Tên món": "Tên món",
        "Số lượng": "Số lượng",
        "Đơn giá": "Đơn giá",
        "Thành tiền": "Thành tiền"
    },
]
'''
response = model.chat(tokenizer, pixel_values, prompt,
generation_config)

del pixel_values
response

```

Phụ lục 3.4. Triển khai Flask và Expose ra API qua Ngrok

```

# Setup Ngrok Token
from google.colab import userdata
from flask import Flask, jsonify, request
from flask_cors import CORS
from pyngrok import ngrok

authtoken = userdata.get("ngrok_token")
ngrok.set_auth_token(authtoken)

# Initialize Flask app
app = Flask(__name__)
CORS(app)

```

```

prompt = '''<image>\nNhận diện hoá đơn trong ảnh. Chỉ trả về phần
liệt kê các mặt hàng hàng dưới dạng CSV'''

@app.route('/ocr', methods=['POST'])
def index():
    data = request.json
    image_url = data.get('image_url', None)

    response_message = ocr_by_llm(image_url, prompt)

    return jsonify({
        "response_message": response_message
    })

def ocr_by_llm(image_url, prompt):

    pixel_values = load_image(image_url,
max_num=6).to(torch.bfloat16).cuda()

    response_message = model.chat(tokenizer, pixel_values, prompt,
generation_config)

    del pixel_values

    print(response_message)
    return response_message

if __name__ == '__main__':

    ngrok_url = ngrok.connect(5555)
    print(ngrok_url)

    app.run(port=5555)

```